



AWS Application Programming Interface (API) and Command Line Interface (CLI)

Lecture Slides

These lecture slides are provided for personal and non-commercial use only

Please do not redistribute or upload these lecture slides elsewhere.

Good luck on your exam!

AWS Application Programming Interface (API)

What is an Application Programming Interface (API)?

An API is software that allows two applications/services to communicate with each other.
The most common type of API is via HTTP/S requests.

AWS API is an HTTP API and you interact by sending HTTPS requests,
using an application interacting with APIs like **Postman**.



Each AWS Service has its own **Service Endpoint** which you send requests

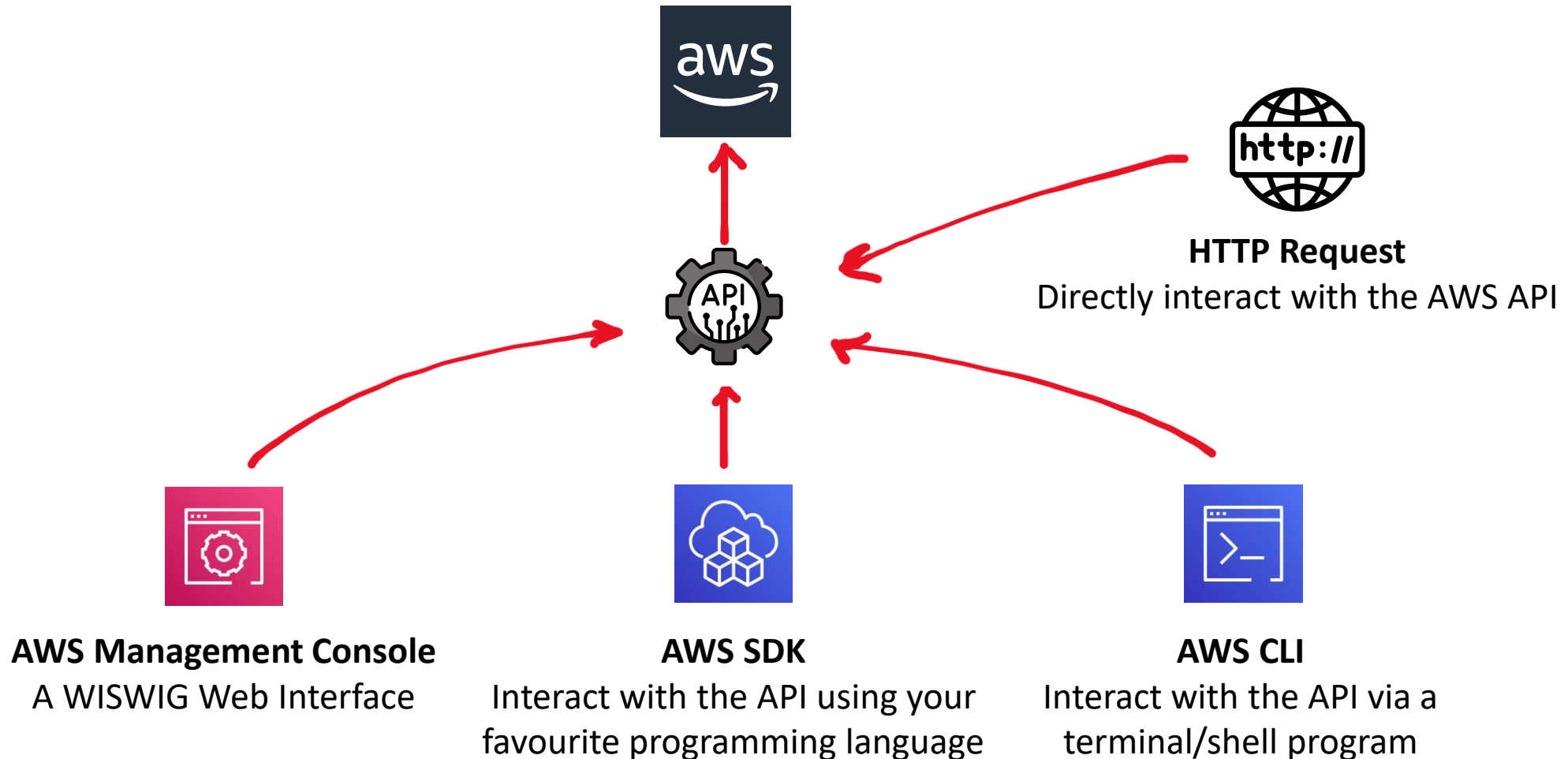
GET / HTTP/1.1
host: **monitoring.us-east-1.amazonaws.com**
x-amz-target: GraniteServiceVersion20100801.GetMetricData
x-amz-date: 20180112T092034Z
Authorization: **AWS4-HMAC-SHA256 Credential=REDACTEDREDACTED/20180411/....**
Content-Type: application/json
Accept: application/json
Content-Encoding: amz-1.0
Content-Length: 45
Connection: keep-alive

To authorize use you will need generate a **signed request**
You make a separate request with your AWS credentials and get back a token.

You need to also provide an **ACTION**
and accompanying **parameters** as the payload

AWS Application Programming Interface (API)

Rarely do users directly send HTTP requests directly to the AWS API.
It's much easier to interact with the API via a variety of Developer Tools



AWS Command Line Interface (CLI)

What is a CLI?

A Command Line Interface (CLI) **processes commands to a computer program in the form of lines of text.**

Operating systems implement a command-line interface in a shell.

What is a Terminal?

A terminal is a text only interface (input/output environment)

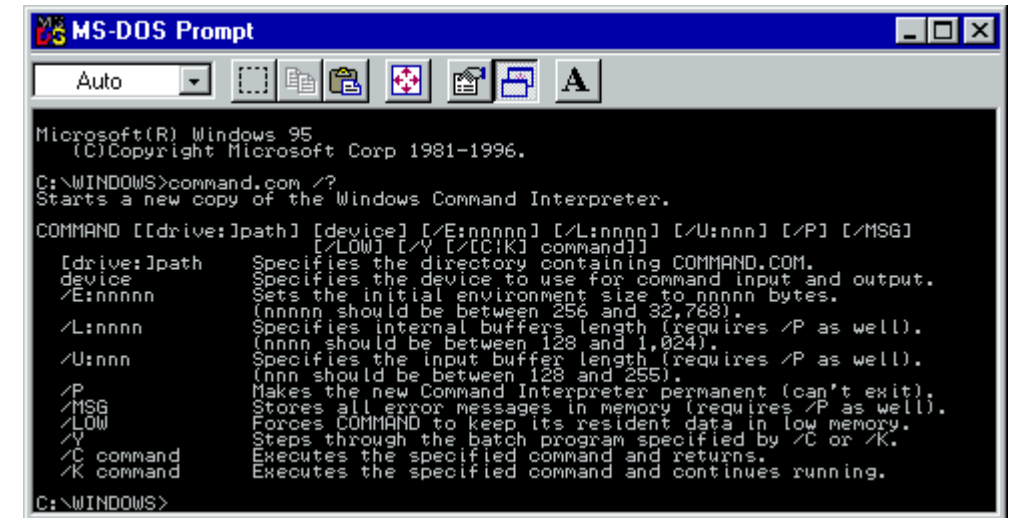
What is a Console?

A console is a physical computer to physically input information into a terminal

What is a Shell?

A shell is the command line program that users interact with to input commands. Popular shell programs:

- **Bash**
- Zsh
- PowerShell



```
MS-DOS Prompt
Auto
Microsoft(R) Windows 95
(C)Copyright Microsoft Corp 1981-1996.
C:\WINDOWS>command.com /?
Starts a new copy of the Windows Command Interpreter.
COMMAND [[drive:]path] [device] [/E:nnnnn] [/L:nnnn] [/U:nnn] [/P] [/MSG]
[/LOW] [/V [/C[K] command]]
[drive:]path Specifies the directory containing COMMAND.COM.
device Specifies the device to use for command input and output.
/E:nnnnn Sets the initial environment size to nnnnn bytes.
(nnnnn should be between 256 and 32,768).
/L:nnnn Specifies internal buffers length (requires /P as well).
(nnnn should be between 128 and 1,024).
/U:nnn Specifies the input buffer length (requires /P as well).
(nnn should be between 128 and 255).
/P Makes the new Command Interpreter permanent (can't exit).
/MSG Stores all error messages in memory (requires /P as well).
/LOW Forces COMMAND to keep its resident data in low memory.
/V Steps through the batch program specified by /C or /K.
/C command Executes the specified command and returns.
/K command Executes the specified command and continues running.
C:\WINDOWS>
```

People commonly (erroneously) use **Terminal, Shell or Console** to generally describe interacting with a Shell.

AWS Command Line Interface (CLI)



AWS Command Line Interface (CLI) allows users to programmatically interact with the AWS API via entering **single or multi-line commands** into a shell or terminal

```
aws ec2 describe-instances \
  --filters Name=tag-key,Values=Name \
  --query 'Reservations[*].Instances[*].
{Instance:InstanceId,AZ:Placement.AvailabilityZone,Name:Tags[?
Key==`Name`][0].Value}' \
  --output table
```

DescribeInstances		
AZ	Instance	Name
us-east-2b	i-057750d42936e468a	my-prod-server
us-east-2a	i-001efd250faaa6ffa	test-server-1
us-east-2a	i-027552a73f021f3bd	test-server-2



The AWS CLI is a Python executable program.

- Python is required to install AWS CLI

The AWS CLI can be installed on Windows, Mac or Linux/Unix

The name of the CLI program is **aws**

Access Keys

Access Keys is a **key and secret** required to have programmatic access to AWS resources when interacting with the AWS API outside of the AWS Management Console



An Access Key is commonly referred to as **AWS Credentials**

A user must be **granted access** to use Access Keys

Select AWS credential type*

☒ **Access key - Programmatic access**
Enables an **access key ID** and **secret access key** for the AWS API, CLI, SDK, and other development tools.

☐ **Password - AWS Management Console access**
Enables a **password** that allows users to sign-in to the AWS Management Console.

- Never share your access keys
- Never commit access keys to a codebase
- You can have two active Access Keys
- You can deactivate Access Keys
- Access Keys have whatever access a user has to AWS resources.

Generate an Access Key and Secret

Access key ID	Secret access key
AKIAZRJIQN2ODG55TBXO	jzXt1gj1PE1f/y9k5JVI2TnwvQ6CSwanzg8aUP3O Hide

Access Keys

Access Keys are to be store in `~/.aws/credentials` and follow a TOML file format

Default will be the access key  used when no profile is specified.

You can store multiple access  keys by giving the **profile** names.

```
[default]
aws_access_key_id=AKIAIOSFODNN7EXAMPLE
aws_secret_access_key=wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
[exampro]
aws_access_key_id=AKIAIOSFODNN7EXAMPLE
aws_secret_access_key=wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
region=ca-central-1
```

You can use the **aws configure**  CLI command to populate the credential file.

```
$ aws configure
AWS Access Key ID [None]: AKIAIOSFODNN7EXAMPLE
AWS Secret Access Key [None]: wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
Default region name [None]: us-west-2
Default output format [None]: json
```

The AWS SDK and CLI will automatically read from these environment variables.

This is the safe way of using an Access Key within your code.

```
$ export AWS_ACCESS_KEY_ID=AKIAIOSFODNN7EXAMPLE
$ export AWS_SECRET_ACCESS_KEY=wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
$ export AWS_DEFAULT_REGION=us-west-2
```

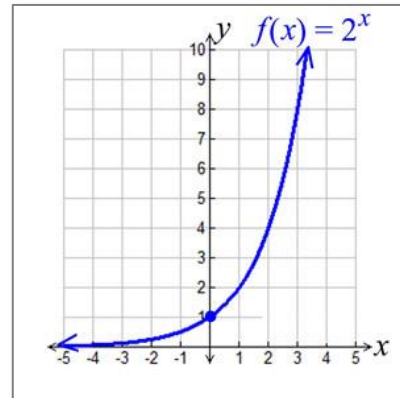
API Retries and Exponential Backoff

When interacting with APIs over a network, **it's common for a networking issue to occur** due to the number of devices a request has to pass through and point of failures:

- DNS Servers
- Switches
- Load Balancers

When working with APIs you need to plan for possible network failure **by trying again**.

Exponential	Factor	Standard Form
$2^2 =$	$2 \times 2 =$	4
$2^3 =$	$2 \times 2 \times 2 =$	8
$2^4 =$	$2 \times 2 \times 2 =$	16
$2^5 =$	$2 \times 2 \times 2 =$	32



It is industry wide recommended for APIs to use an **exponential backoff** before trying again.

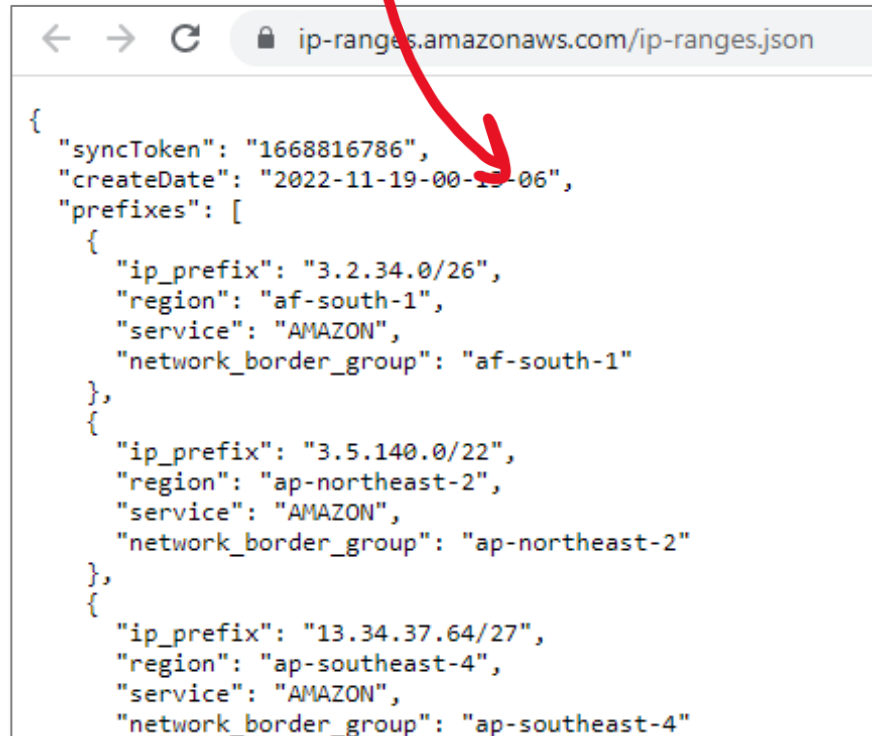
- Try again in 1 second
- Try again in 2 seconds
- Try again in 4 seconds
- Try again in 8 seconds
- Try again in 16 seconds
- Try again in 32 seconds

A good CLI or SDK will have exponential backoff built in with options like how many times it should try.

AWS IP Address ranges

AWS Publishes all of its IP Address Ranges at this url in json format:

<https://ip-ranges.amazonaws.com/ip-ranges.json>

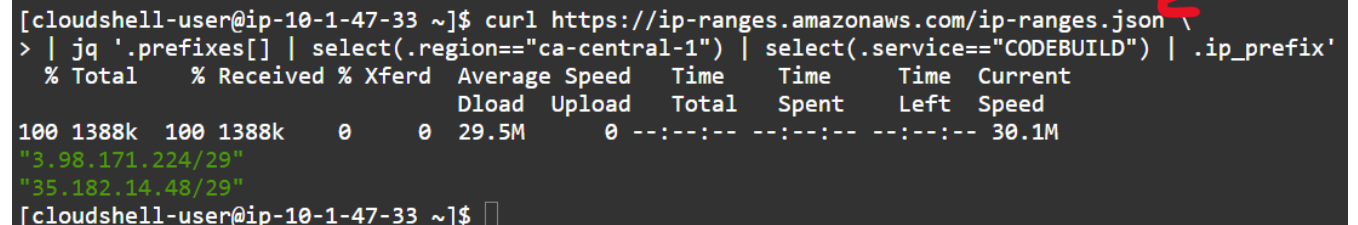


```
{
  "syncToken": "1668816786",
  "createDate": "2022-11-19-00-15-06",
  "prefixes": [
    {
      "ip_prefix": "3.2.34.0/26",
      "region": "af-south-1",
      "service": "AMAZON",
      "network_border_group": "af-south-1"
    },
    {
      "ip_prefix": "3.5.140.0/22",
      "region": "ap-northeast-2",
      "service": "AMAZON",
      "network_border_group": "ap-northeast-2"
    },
    {
      "ip_prefix": "13.34.37.64/27",
      "region": "ap-southeast-4",
      "service": "AMAZON",
      "network_border_group": "ap-southeast-4"
    }
  ]
}
```

Organizations might want these IP Ranges in case they need to configure their internal networking devices to allowlist these ranges.

Since it's a json endpoint they can programmatically pull the json data.

```
curl https://ip-ranges.amazonaws.com/ip-ranges.json \
| jq '.prefixes[] | select(.region=="ca-central-1") |
select(.service=="CODEBUILD") | .ip_prefix'
```



```
[cloudshell-user@ip-10-1-47-33 ~]$ curl https://ip-ranges.amazonaws.com/ip-ranges.json \
> | jq '.prefixes[] | select(.region=="ca-central-1") | select(.service=="CODEBUILD") | .ip_prefix'
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total     Spent    Left  Speed
100 1388k  100 1388k    0     0  29.5M      0  --:--:-- --:--:-- --:--:-- 30.1M
"3.98.171.224/29"
"35.182.14.48/29"
[cloudshell-user@ip-10-1-47-33 ~]$
```

Smithy



Smithy 2.0 is AWS's open-source Interface Definition Language (IDL) for web services.

- **Smithy** is a language for defining services and SDKs.
- Smithy and its server generator unlocks model-first development.
- forces you to define your interface first rather than let your API to become implicitly defined by your implementation choices

```
// Define the namespace
namespace com.amazonaws.s3

// Define a simplified S3 service
service SimpleS3 {
    version: "2023-12-21",
    operations: [ListBuckets, PutObject]
}

// Define an operation to list buckets
operation ListBuckets {
    output: ListBucketsOutput
}

// Define the output structure for ListBuckets
operation
structure ListBucketsOutput {
    buckets: BucketList
}

// Define a list of buckets
list BucketList {
    member: Bucket
}
```

Security Token Service (STS)

STS is a web service that enables you to request **temporary, limited-privilege credentials** for IAM users or federated users

AWS Security Token Service (STS) is a **global service**, and all AWS STS requests go to a single endpoint at <https://sts.amazonaws.com>

An STS will return:

- **AccessKeyID**
- **SecretAccessKey**
- **SessionToken**
- Expiration

You can use the following API actions to obtain STS:

- **AssumeRole**
- AssumeRoleWithSAML
- **AssumeRoleWithWebIdentity**
- DecodeAuthorizationMessage
- GetAccessKeyInfo
- GetCallerIdentity
- GetFederationToken
- GetSessionToken

Security Token Service (STS)

We **assume a role via STS** and optionally provide a session name

```
{
  "Credentials": {
    "AccessKeyId": "ASIA...IAQ",
    "SecretAccessKey": "wJalr...zuE3",
    "SessionToken": "FQoGz...<snipped>...zpZC",
    "Expiration": "2023-12-21T00:00:00Z"
  },
  "AssumedRoleUser": {
    "AssumedRoleId": "AROA...session1",
    "Arn": "arn:aws:sts::123456789012:..."
  }
}
```

It returns back **credentials**

```
import boto3

# assume a role and get temporary credentials
sts_client = boto3.client('sts')
response = sts_client.assume_role(
    RoleArn='arn:aws:iam::123456789012:role/demo',
    RoleSessionName='session1'
)
creds = response['Credentials']

# load in temporary credentials
s3 = boto3.client('s3',
    aws_access_key_id=creds['AccessKeyId'],
    aws_secret_access_key=creds['SecretAccessKey'],
    aws_session_token=creds['SessionToken']
)
```

Load them into the **client**

Security Token Service (STS)

We can **generate out temporary credentials** via STS with the CLI

```
aws sts assume-role \  
--role-arn arn:aws:iam::123456789012:role/my-role \  
--role-session-name mySession
```

```
export AWS_ACCESS_KEY_ID="ASIA...IAQ"  
export AWS_SECRET_ACCESS_KEY="wJalr...zuE3"  
export AWS_SESSION_TOKEN="FQoGz...<snipped>...zpZC"
```

```
{  
  "Credentials": {  
    "AccessKeyId": "ASIA...IAQ",  
    "SecretAccessKey": "wJalr...zuE3",  
    "SessionToken": "FQoGz...<snipped>...zpZC",  
    "Expiration": "2023-12-21T00:00:00Z"  
  },  
  "AssumedRoleUser": {  
    "AssumedRoleId": "AROA...session1",  
    "Arn": "arn:aws:sts::123456789012:..."  
  }  
}
```

Then we can **load them** into our CLI configuration files or in the environment variables.

Signing AWS API Requests

When you send API requests to AWS, **you sign the requests** so that **AWS can identify who sent them**

When you use the AWS CLI or AWS SDK, **requests are signed for you automatically.**

Some requests don't need to be signed:

- Anonymous requests to Amazon S3
- Some API operations to STS e.g. AssumeRoleWithWebIdentity

Signatures do the following:

- Prevent data tampering
- Verifies the identity of the requester

```
https://s3.amazonaws.com/examplebucket/test.txt
?X-Amz-Algorithm=AWS4-HMAC-SHA256
&X-Amz-Credential=<your-access-key-id>/20130721/us-...
&X-Amz-Date=20130721T201207Z
&X-Amz-Expires=86400
&X-Amz-SignedHeaders=host
&X-Amz-Signature=<signature-value>
```

Example of a signature being
supplied in a Query Parameter



AWS has two different protocols for signing:

- AWS Signature Version 2 (only used in legacy customer's usecases)
- **AWS Signature Version 4**

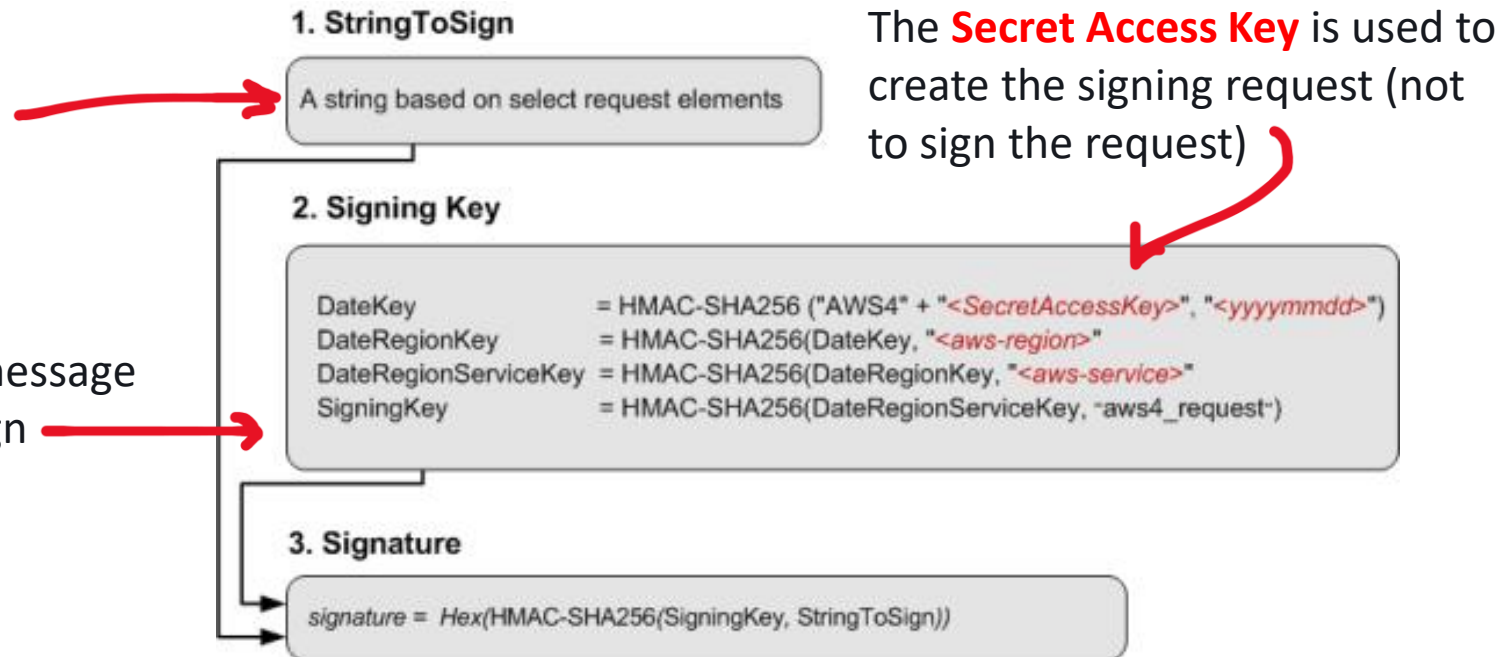
AWS Signature Version 4

Step 1 Generate String to Sign

concatenate select request elements to form a string, referred to as the *string to sign*

Step 2 Create a Signing Key

use a signing key to calculate the hash-based message authentication code (HMAC) of the string to sign



The process to sign changes based on the request type

Request Types:

- **Authorization Header (the most common way)**
- A Post Request
- Query parameters

Service Endpoints

To connect programmatically to an AWS service, you use an endpoint.

An endpoint is the URL of the entry point for an AWS web service.

This is generally the format of the a service endpoint (it varies per service)

Generally TLSv2 is expected some older APIs might support TLSv1, TLSv1.1, or HTTP.

`protocol://service-code.region-code.amazonaws.com`

`https://cloudformation.us-east-2.amazonaws.com`

There are four types of Service Endpoints:

1. **Global Endpoints** – AWS Services that use the same endpoint
 2. **Regional Endpoints** – AWS Services require you to specify a region
 3. **FIPS Endpoints** – Some endpoints support FIPS for enterprise use
 4. **Dualstack Endpoints** – Allows IPV6 or IPV4
- The types of Service Endpoints can be combined eg Regional + FIPS + Dualstack
 - An AWS Service may have multiple different service endpoints.
 - AWS CLI and AWS SDK will automatically use the default endpoint for each service in an AWS Region

Service Endpoints – Regional Endpoints

A Region Endpoint **requires* a regional code.
Regional AWS Services have regional endpoints

```
protocol://service-code.region-code.amazonaws.com
```

Examples of AWS Services
that use **regional endpoints**:

- EC2
- S3
- CodeBuild
- SES
- SageMaker
- Lambda
- ECS
- AWS Config
- CloudTrail
- And many more....



```
codebuild.us-east-2.amazonaws.com  
codebuild.ca-central-1.amazonaws.com  
ec2.us-east-1.amazonaws.com  
s3.eu-south-1.amazonaws.com
```

Some regional services will accept a global-like endpoint that will route to the default region. (The default for all API calls is us-east-1) e.g.



```
ec2.amazonaws.com
```

Service Endpoints – Global Endpoints

A Global Endpoint does not specific a region
Global AWS Services have global endpoints

Examples of AWS Services
that use **global endpoints**:

- Route53
- IAM
- Global Accelerator
- CloudFront
- ...

```
protocol://service-code.amazonaws.com
```

```
route53.amazonaws.com  
iam.amazonaws.com  
globalaccelerator.amazonaws.com  
cloudfront.amazonaws.com
```

In the documentation global endpoints still indicate a region.

CloudFront lists only us-east-1, **Route53** lists the same endpoint for all regions



Region Name	Region	Endpoint	Protocol
US East (N. Virginia) Region	us-east-1	cloudfront.amazonaws.com	HTTPS
		cloudfront-fips.amazonaws.com	HTTPS



Region Name	Region	Endpoint	Protocol
US East (Ohio)	us-east-2	route53.amazonaws.com	HTTPS
US East (N. Virginia)	us-east-1	route53.amazonaws.com	HTTPS
US West (N. California)	us-west-1	route53.amazonaws.com	HTTPS

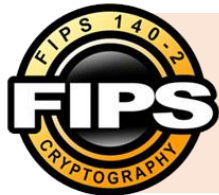
Service Endpoints – FIPS Endpoints

A FIPS Endpoint is regional or global endpoint that supports FIPS.
FIPS endpoints are available for specific services and specific regions

```
ec2-fips.us-east-2.amazonaws.com  
api-fips.sagemaker.us-east-2.amazonaws.com  
cloudfront-fips.amazonaws.com
```

The format of a FIPS endpoint will vary

Using a FIPS endpoint may be required by enterprises that interact with the US government



Federal Information Processing Standard (FIPS) Publication 140-2 is a US and Canadian government standard that **specifies the security requirements for cryptographic modules** that protect sensitive information.

Using the **AWS CLI** you'd specify the endpoint URL:

```
aws kms create-key --endpoint-url https://kms-fips.us-west-2.amazonaws.com
```

Using the **AWS SDK** you specify the Environment Variable: **AWS_USE_FIPS_ENDPOINT=true**

Service Endpoints – Troubleshooting Endpoints

When **troubleshooting** service endpoints we can use telnet, nslookup and ping:



```
telnet s3.ap-southeast-2.amazonaws.com 443
nslookup s3.amazonaws.com
ping s3.amazonaws.com
```

telnet provides a command line interface for communication with a remote device or server.

- Can we establish a connection?

```
[cloudshell-user@ip-10-1-47-33 ~]$ telnet s3.ap-southeast-2.amazonaws.com 443
Trying 52.95.128.31...
Connected to s3.ap-southeast-2.amazonaws.com.
Escape character is '^]'.
```

nslookup queries internet domain name servers

- Is there a routable path?

```
[cloudshell-user@ip-10-1-47-33 ~]$ nslookup s3.amazonaws.com
Server:          127.0.0.1
Address:         127.0.0.1#53

Non-authoritative answer:
Name:   s3.amazonaws.com
Address: 52.217.68.150
Name:   s3.amazonaws.com
Address: 52.217.163.176
Name:   s3.amazonaws.com
Address: 52.217.88.110
```

ping test and verify if a particular destination address exists and can accept requests

- We sent it something, did it send anything back?

```
gitpod / $ ping s3.amazonaws.com
PING s3.amazonaws.com (52.216.82.83) 56(84) bytes of data.
64 bytes from s3-1.amazonaws.com (52.216.82.83): icmp_seq=1 ttl=239 time=62.3 ms
64 bytes from s3-1.amazonaws.com (52.216.82.83): icmp_seq=2 ttl=239 time=61.6 ms
```

Service Endpoints – Dualstack Endpoints

A Dual-stack endpoint supports both **IPv4** and **IPv6** traffic

S3 requires you to specify the bucket,
and can be done in either

- Virtual hosted Style format → `bucketname.s3.dualstack.us-east-1.amazonaws.com`
- Path-style format → `s3.dualstack.us-east-1.amazonaws.com/bucketname`

EC2 uses the “api.aws”
domain for dual-stack:

→ `ec2.us-east-1.api.aws`

How to use a dual-stack endpoint via the AWS
CLI (*you need to specify region flag as well*)

→

```
aws ec2 describe-regions \  
--region us-east-2 \  
--endpoint-url https://ec2.us-east-2.api.aws
```

- There does not appear to be many dualstack endpoints
- Dualstack endpoints do not seem to appear on global endpoints

Service Endpoints – Feature Specific Endpoints

Some AWS Services have feature-specific endpoints
with quite the varied convention

```
route53.amazonaws.com  
route53domains.us-east-1.amazonaws.com  
route53resolver.us-east-2.amazonaws.com
```

```
s3.us-east-2.amazonaws.com  
s3-accesspoint.us-east-2.amazonaws.com  
<aws-account-id>.s3-control.us-east-2.amazonaws.com
```

```
api.sagemaker.sa-east-1.amazonaws.com  
runtime.sagemaker.us-east-2.amazonaws.com  
edge.sagemaker.us-east-2.amazonaws.com
```

API Actions (Operations)

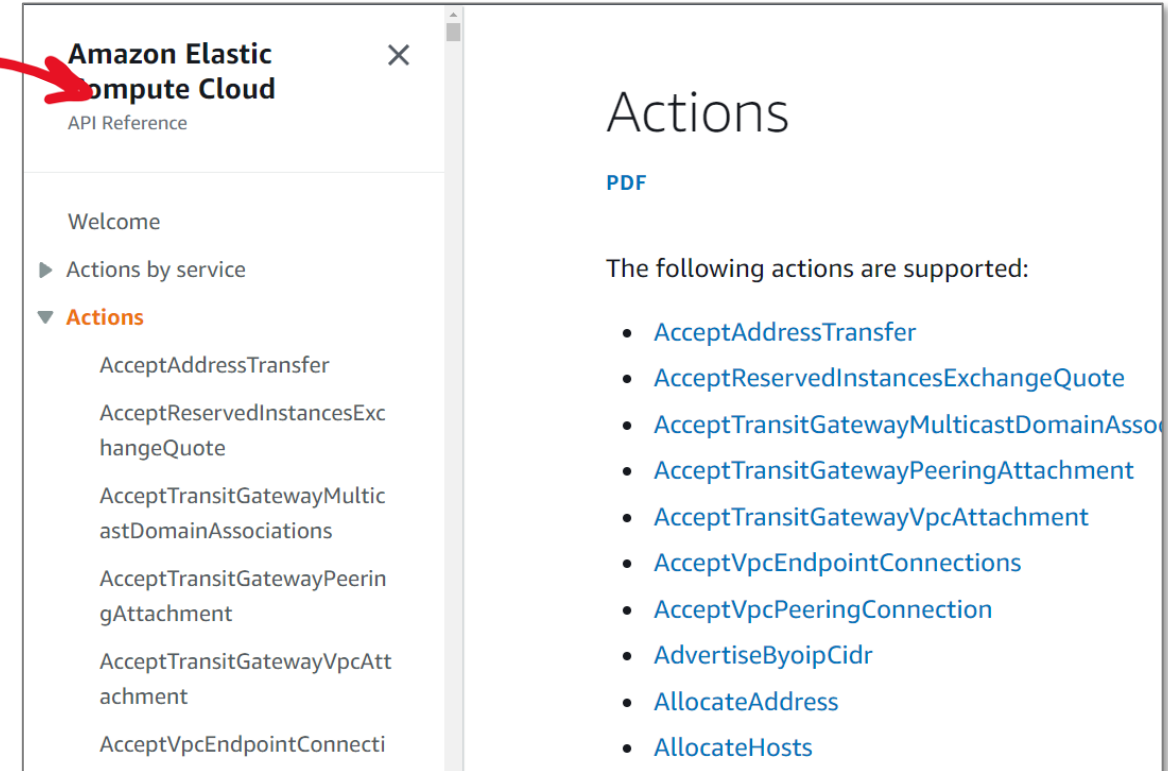
Each AWS Service has an API Reference and within that a **list of all possible API Actions** (aka API operations)

When a service-endpoint is called you need to **specific the action**.

```
https://ec2.amazonaws.com/?Action=RunInstances
```

The naming of AWS API Actions is all over the place:

- **Common**
 - **Describe<>, Create<>, Update<>, Delete<>**
 - Describe can be a list or detailed about a resource
 - aws ec2 describe-vpcs
 - aws ec2 describe-vpc
- **Uncommon**
 - List<>, Put<>, Get<>, Copy<>, Modify<> ... and probably more
- **Very Uncommon**
 - Disable<>, Enable<>, Associate<>, Accept<>, Allow<>, Cancel<> and many more



API Actions (Operations)

The **AWS CLI** will use the same action name

```
aws ec2 run-instances \  
  --image-id ami-0abcdef1234567890 \  
  --instance-type t2.micro \  
  --key-name MyKeyPair
```

The **AWS SDK** will use the same action name

```
resp = client.run_instances({  
    block_device_mappings: [  
        {  
            device_name: "/dev/sdh",  
            ebs: {  
                volume_size: 100,  
            },  
        },  
    ],  
    image_id: "ami-abc12345"
```

AWS IAM Policies will use
the same action name

```
{  
    "Version": "2012-10-17",  
    "Statement": [  
        {  
            "Effect": "Allow",  
            "Action": [  
                "ec2:RunInstances",  
                "ec2:TerminateInstances",  
                "ec2:StopInstances",
```

AWS CLI Command Reference

The AWS CLI Command Reference is a **website** that contains detailed documentation on commands, subcommands and their parameters

<https://awscli.amazonaws.com/v2/documentation/api/latest/index.html>

The image displays two screenshots of the AWS CLI Command Reference website. The left screenshot shows the main index page, which includes the AWS logo, a 'Quick search' bar, a 'Feedback' section, and a list of command references under the heading 'Command Reference'. The right screenshot shows the 'create-project' command page, which includes a 'Table of Contents' with links to 'Description', 'Synopsis', 'Options', 'Global Options', 'Examples', and 'Output'. The 'Description' section states 'Creates a build project.' and 'See also: AWS API Documentation'. The 'Synopsis' section shows the command syntax: `create-project --name <value> [--description <value>] [--source <value>] [--secondary-sources <value>] [--source-version <value>] [--secondary-source-versions <value>] --artifacts <value>`.

AWS CLI – Return Codes

When an AWS CLI command is run it has a return code that can be **observed in the terminal**.



```
echo $? # Bash  
echo $lastexitcode # Power Shell
```

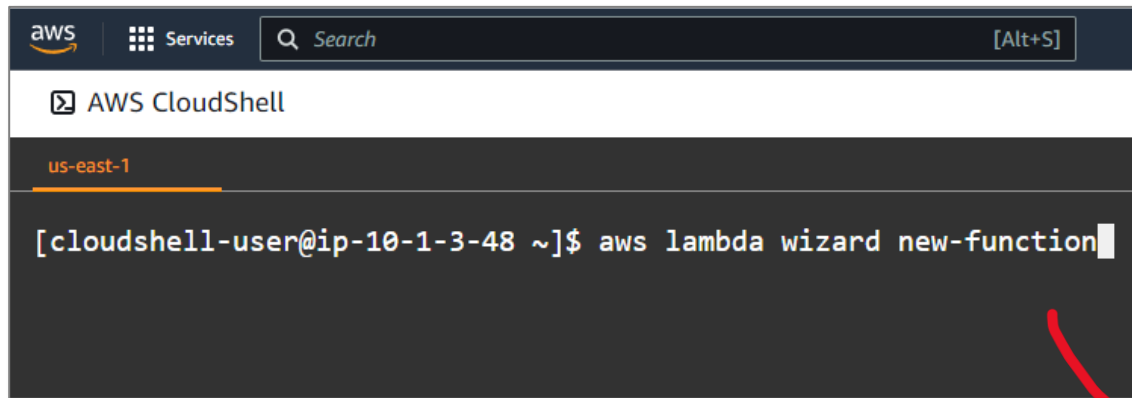
A red arrow points from the text 'observed in the terminal' to the terminal screenshot.

Code	Meaning
0	200 Status code was return by AWS API API. No error generated.
1	One more Amazon S3 transfer operations failed
2	The command entered couldn't be parsed e.g. missing required arguments
2 (S3)	One or more files marked for transfer were skipped during transfer process e.g. file doesn't exist
130	The command was interrupted by a SIGINT e.g. Ctrl + C
252	Command syntax was invalid (unknown or incorrect parameters)
253	The system environment or configuration was invalid eg. Bad or missing AWS Credentials
254	Command was successful sending a request, Error return by the API from the AWS Service
255	The command failed. Could be from the AWS CLI or the AWS Service

AWS CLI – Wizards

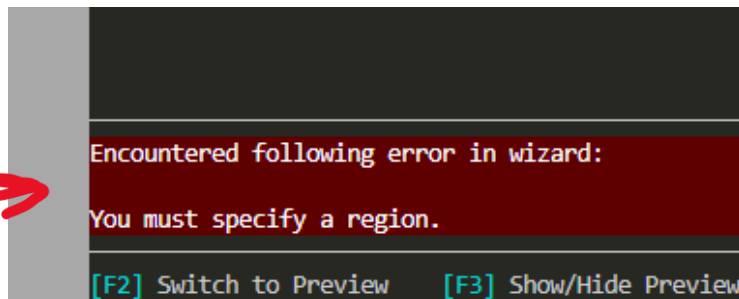
AWS CLI has a special **wizard** command for very specific services that allows you to go through a wizard process.

- aws **configure** wizard
- aws **dynamodb** wizard **new-table**
- aws **events** wizard **new-rule**
- aws **iam** wizard **new-role**
- aws **lambda** wizard **new-function**



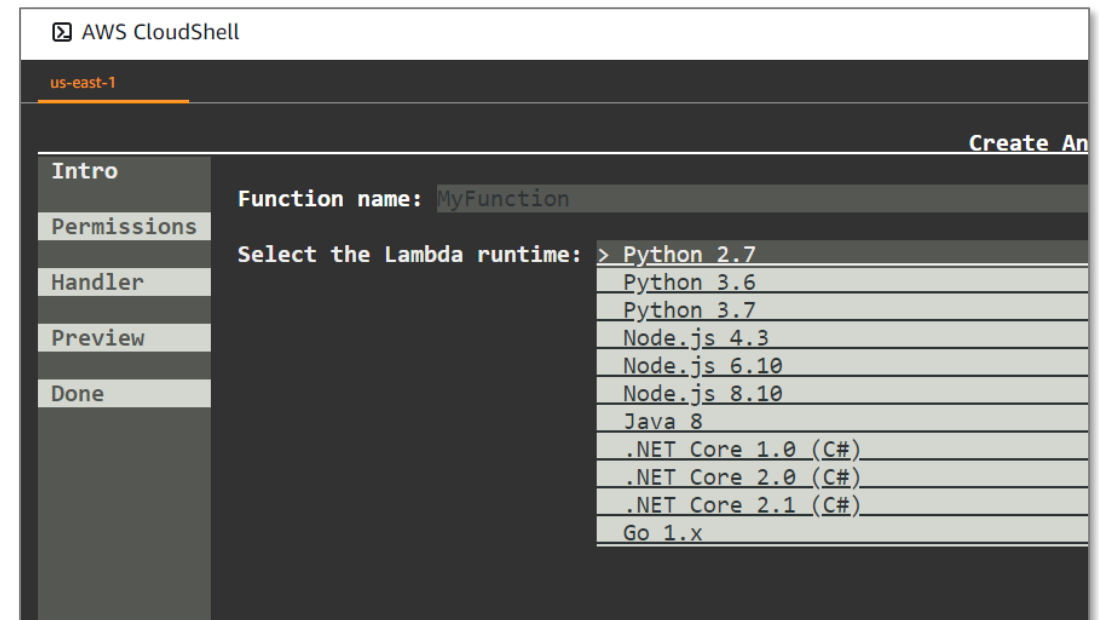
```
aws CloudShell
us-east-1
[cloudshell-user@ip-10-1-3-48 ~]$ aws lambda wizard new-function
```

When you enter the wizard it will prompt you with a GUI with multiple steps to easily provision



```
Encountered following error in wizard:
You must specify a region.
[F2] Switch to Preview [F3] Show/Hide Preview
```

The wizard will fail if you don't have a default region set:
`export AWS_DEFAULT_REGION=us-west-2`



```
aws CloudShell
us-east-1
Create An

Intro
Permissions
Handler
Preview
Done

Function name: MyFunction

Select the Lambda runtime: > Python 2.7
Python 3.6
Python 3.7
Node.js 4.3
Node.js 6.10
Node.js 8.10
Java 8
.NET Core 1.0 (C#)
.NET Core 2.0 (C#)
.NET Core 2.1 (C#)
Go 1.x
```

AWS CLI – Output Formats

AWS CLI allows you to specify the output of AWS CLI command using the **--output** flag.

The following outputs supported: **json, yaml, yaml-stream, text, table**

```
aws ec2 describe-instances --output yaml
```

```
- InstanceId: i-1234567890abcdef0
  ImageId: ami-05b3bc2c6fbb23456
  InstanceType: t2.micro
  ...
```

```
aws s3 ls s3://mybucket --output json
```

```
[
  {
    "Key": "file1.txt",
    "LastModified": "2023-01-01T12:00:00.000Z",
    "Size": 1234
  },
  ...
]
```

```
aws dynamodb list-tables --output table
```

ListTables
TableName
Table1
Table2

```
aws iam list-users --output text
```

```
USERS    arn:aws:iam::123456789012:user/User1    User1
USERS    arn:aws:iam::123456789012:user/User2    User2
USERS    arn:aws:iam::123456789012:user/User3    User3
```

You can also set a default output in the config file or set an environment variable eg. **AWS_DEFAULT_OUTPUT**

AWS CLI – Pagination

Pagination is used when you have too much data, so you break it up into sections which we call pages, and then you specify which page of data you want.

```
aws s3api list-objects \  
  --bucket my-bucket \  
  --no-paginate
```

```
aws s3api list-objects \  
  --bucket my-bucket \  
  --page-size 100
```

```
aws s3api list-objects \  
  --bucket my-bucket \  
  --max-items 100
```

```
aws s3api list-objects \  
  --bucket my-bucket \  
  --max-items 100 \  
  --starting-token eyJNYXJrZXIiOiBu...
```

Server-side flags:

```
--no-paginate  
--page-size  
--max-items  
--starting-token
```

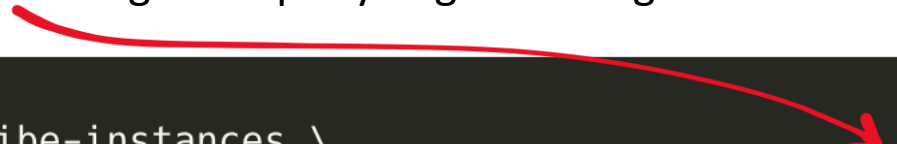
Pagination for the AWS CLI can be done **two ways**:

- Server Side
- Client Side

Client Side pagination

We can **paginate** using the `--query` flag and using JMESPATH

```
aws ec2 describe-instances \  
  --query 'Reservations[*].Instances[*].InstanceId | [0:5]'
```



AWS CLI – Command Structure

The “toplevel” **command** points to an AWS Service
eg. ec2, s3 dynamodb, ses

The **subcommand** correlates to an API action.
eg. describe-tables

```
aws <command> <subcommand> [options and parameters]
```

```
aws ec2 run-instances \  
  --image-id ami-0abcdef1234567890 \  
  --instance-type t2.micro \  
  --key-name MyKeyPair
```

Commands map to **service endpoints**

Subcommands map to **API actions** (operations)

```
https://ec2.amazonaws.com/?Action=RunInstances
```

What is Command Flags?



Shell programs will often support **command flags** to pass options or parameters. Programs may call their flags by the different names: options, parameters or fields.

```
curl https://v.gd --output index.html
curl https://v.gd --output=index.html
wget --output index.html https://v.gd
wget -o index.html https://v.gd
wget -O index.html https://v.gd
npm install -g
ls -l --all
ls -la
```

Flags start with “--” and *usually* expect an input

Programs might allow either a space or = between the flag and input

Flags might expected to come first or last

Flags can have shorthand flags noted by single letter and “-”

There can be captized shorthand flags that do something different

Flags might not require or expect input

You can have multiple flags, where the order doesn’t matter

Some programs let you combine shorthand flags together

AWS CLI – Command Flags

Commands Flags is how **options and parameters** are supplied to an API action when using the AWS CLI.



```
aws <command> <subcommand> [options and parameters]
```

Command Flags for the AWS CLI **are expected to always be written after the subcommand**

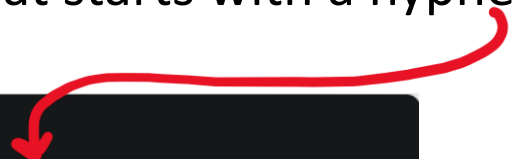


```
aws s3 mb --bucket-name s3://my-bucket
```

- AWS does not have shorthand command flag
- The order of flags does not matter, they simply must appear after the subcommand
- Some flags do not expect an input
- You can use an = or a space between the flag key and value
- Sometimes you need to wrap the value of flag in quotations to avoid input side affects

AWS CLI – Command Flags

With command flag for AWS CLI you can use either a space or an equal (=)
Using equal is required if you have an input that starts with a hyphen (-)



```
aws ec2 delete-key-pair --key-name=-mykey
```

AWS CLI – Wait Commands

The **wait** subcommand will **wait for a particular condition is satisfied**.
This is useful for long running API Actions e.g. stopping an instance or deleting a volume
The wait subcommand is only supported for specific AWS API commands.

You place wait in-front of the supported subcommand



```
aws ec2 wait vpc-available \
  --vpc-ids vpc-1234567890abcdef0
```

Available Commands that work with Wait

- | | | | |
|------------------------------|---------------------------|-----------------------------------|----------------------------------|
| • bundle-task-complete | • instance-exists | • nat-gateway-deleted | • volume-available |
| • conversion-task-cancelled | • instance-running | • network-interface-available | • volume-in-use |
| • conversion-task-completed | • instance-status-ok | • password-data-available | • vpc-available |
| • conversion-task-deleted | • instance-stopped | • security-group-exists | • vpc-exists |
| • customer-gateway-available | • instance-terminated | • snapshot-completed | • vpc-peering-connection-deleted |
| • export-task-cancelled | • internet-gateway-exists | • spot-instance-request-fulfilled | • vpc-peering-connection-exists |
| • export-task-completed | • key-pair-exists | • subnet-available | • vpn-connection-available |
| • image-available | • nat-gateway-available | • system-status-ok | • vpn-connection-deleted |
| • image-exists | | | • volume-deleted |

AWS CLI – Aliases

AWS CLI Aliases allow you to **shorten commonly used commands** similar to how aliases work for the bash terminal.

All commands appear under the **toplevel** group

To configure you need to create a file at `~/.aws/cli/alias`

```
[toplevel]
amazon-linux-amis = ec2 describe-images \
  --filter \
    Name=owner-alias,Values=amazon \
    Name=name,Values="amzn-ami-hvm-*" \
    Name=architecture,Values=x86_64 \
    Name=virtualization-type,Values=hvm \
    Name=root-device-type,Values=ebs \
    Name=block-device-mapping.volume-type,Values=gp2 \
  --query "reverse(sort_by(Images, &CreationDate))[*].[ImageId,Name,Description]" \
  --output text
```

To **execute** your alias you just use it like a top level command

```
aws amazon-linux-amis
```

AWS CLI will just pick up the file, you don't need load, refresh or activate the file

AWS CLI – Aliases

You **cannot** override existing command flags

```
[toplevel]
hello = ec2 describe-vpcs --output json
```



```
[cloudshell-user@ip-10-1-3-48 ~]$ aws hello --output table
{
  "Vpcs": [
    {
      "CidrBlock": "10.0.0.0/16",
      "DhcpOptionsId": "dopt-f4427b8f",
      "State": "available",
      "VpcId": "vpc-00e6b96dc4210933c",
      "OwnerId": "982383527471",

```

We want to override the output to be a table but it doesn't work, it still return json.

You **can** append new command flags

```
[toplevel]
hello = ec2 describe-vpcs
```



```
[cloudshell-user@ip-10-1-3-48 ~]$ aws hello --output table
+-----+-----+-----+-----+
|                                     Describ
+-----+-----+-----+-----+
|                                     Vpc
+-----+-----+-----+-----+
| CidrBlock | DhcpOptionsId | InstanceTenancy | IsDefault
+-----+-----+-----+-----+
| 10.0.0.0/16 | dopt-f4427b8f | default         | False
+-----+-----+-----+-----+
|                                     CidrBlockAss
+-----+-----+-----+-----+
```

We specific the output to be table and it outputs as a table.

AWS CLI – Aliases

An alias comes into two formats:

1. Simple alias: One-line command that maps to an alias

Alias commands can be **single line**

```
[toplevel]
vpcw2 = ec2 describe-vpcs --query "Vpcs[*].[CidrBlock,VpcId]" --output table --region us-west-2

amazon-linux-amis = ec2 describe-images \
  --filter \
    Name=owner,Values=amazon \
    Name=name,Values="amzn-ami-hvm-*" \
    Name=architecture,Values=x86_64 \
    Name=virtualization-type,Values=hvm \
    Name=root-device-type,Values=ebs \
    Name=block-device-mapping.volume-type,Values=gp2 \
  --query "reverse(sort_by(Images, &CreationDate))[*].[ImageId,Name,Description]" \
  --output text
```



Notice you **exclude** “aws” in front
when writing aliases

Alias commands can be **multi-line**

AWS CLI – Aliases

An alias comes into two formats:

2. Bash alias: An alias that triggers a bash script for more complex logic

You use this script content **if() {};** **f** and write bash scripts as you normally would

```
[toplevel]
bucket =
  !f() {
    aws s3 mb --bucket-name "s3://${1}" --region ${2}
  }; f
```



```
aws bucket my-bucket us-east-1
```

You can optionally define input parameters with **special variables**

AWS CLI – Filtering

AWS allows you for specific subcommands to filter the data server-side
For large datasets server-side filtering **can speed up request only returning the data you need.**



The flag for filtering could be: --filter, --filters, --filter-expression

```
aws ec2 describe-instances --filters  
"Name=instance-state-name,Values=running"  
"Name=instance-type,Values=t2.micro"
```



```
{  
  "Reservations": [{  
    "Instances": [{  
      "InstanceId": "i-123456789...",  
      "InstanceType": "t2.micro",  
      "State": {  
        "Name": "running"  
      }  
    }  
  ]  
},  
  ...  
]
```

--filters (list)

The filters.

- `affinity` - The affinity setting for an instance running on a Dedicated Host.
- `architecture` - The instance architecture (`i386` | `x86_64` | `arm64`).
- `availability-zone` - The Availability Zone of the instance.
- `block-device-mapping.attach-time` - The attach time for an instance, for example, `2010-09-15T17:15:20.000Z` .
- `block-device-mapping.delete-on-termination` - A Boolean value indicating whether a volume is deleted on instance termination.
- `block-device-mapping.device-name` - The device name specified in the block device mapping.

AWS CLI – Querying

Querying allows you to filter json client-side by using the **JMESPATH** Query language.

```
aws ec2 describe-instances \  
--query 'Reservations[*].Instances[*].InstanceId | [0:5]'
```



```
{  
  "Reservations": [{  
    "Instances": [{  
      "InstanceId": "i-1234567890abcdef0",  
      "ImageId": "ami-12345abcde67890fg",  
      "InstanceType": "t2.micro",  
      "State": {  
        "Code": 16,  
        "Name": "running"  
      },  
      "PrivateIpAddress": "192.168.1.1",  
      "PublicIpAddress": "203.0.113.1",  
      "Tags": [{  
        "Key": "Name",  
        "Value": "MyInstance1"  
      }  
    ]  
  }  
]
```



```
[  
  "i-1234567890abcdef0",  
  "i-0987654321fedcba1",  
  "i-11223344556677889",  
  "i-22334455667788990",  
  "i-33445566778899001"  
]
```

Most aws cli subcommands supports the query command

What is JSON?

JSON (JavaScript Object Notation) is a lightweight data-interchange format

- It is easy for humans to read and write.
- It is easy for machines to parse and generate
- It is based on a subset of the JavaScript

```
{
  "starships": {
    "enterprise": {
      registry: [
        "NCC-1701",
        "NCC-1701-B",
        "NCC-1701-C",
        "NCC-1701-D"
      ]
    }
  }
}
```

JSON is built on two structures

1. **A collection of name/value pairs**
 - In other languages: realized as an *object*, record, struct, dictionary, hash table, keyed list, or associative array
2. **An ordered list of values**
 - In other languages: realized as an *array*, vector, list, or sequence

JSON is a **text format** that is completely language independent

What is YAML?



YAML is a “markup language” that uses an **indent-based** syntax

YAML example

```
---
Hello: "World"
Planets:
  Sol:
    - Mercury
    - Venus
    - Earth
    - Mars
    - Jupiter
    - Saturn
    - Uranus
    - Neptune
```

Same
thing

JSON example

```
{
  "Hello": "World",
  "Planets": {
    "Sol": [
      "Mercury",
      "Venus",
      "Earth",
      "Mars",
      "Jupiter",
      "Saturn",
      "Uranus",
      "Neptune"
    ]
  }
}
```



YAML rhymes with camel!



YAML is a **superset** of JSON. JSON is considered **valid* in a YAML file.

YAML is more useful as a semi-structured data document.

What is semi-structured data?

- Semi-structured data is data **that contains fields**.
- The fields don't have to be the same in every entity.
- You only define the fields that you need on a per-entity basis.



YAML files can end either the .yaml or .yml

- Some programs enforce “.yaml” as a convention

Why is it called YAML?

YAML originally stood for ***“Yet Another Markup Language”***

The naming (not the language itself) was a **joke** because so many other markup languages existed and intended to replace existing markup languages as a better alternative: eg. HTML, XML, SGML

- YAML is seen by many as a better alternative to JSON when required to write manually

YAML was again renamed to **“YAML Ain’t Markup Language”**



YAML is used in the name of YAML, essentially creating a recursion or infinite loop.

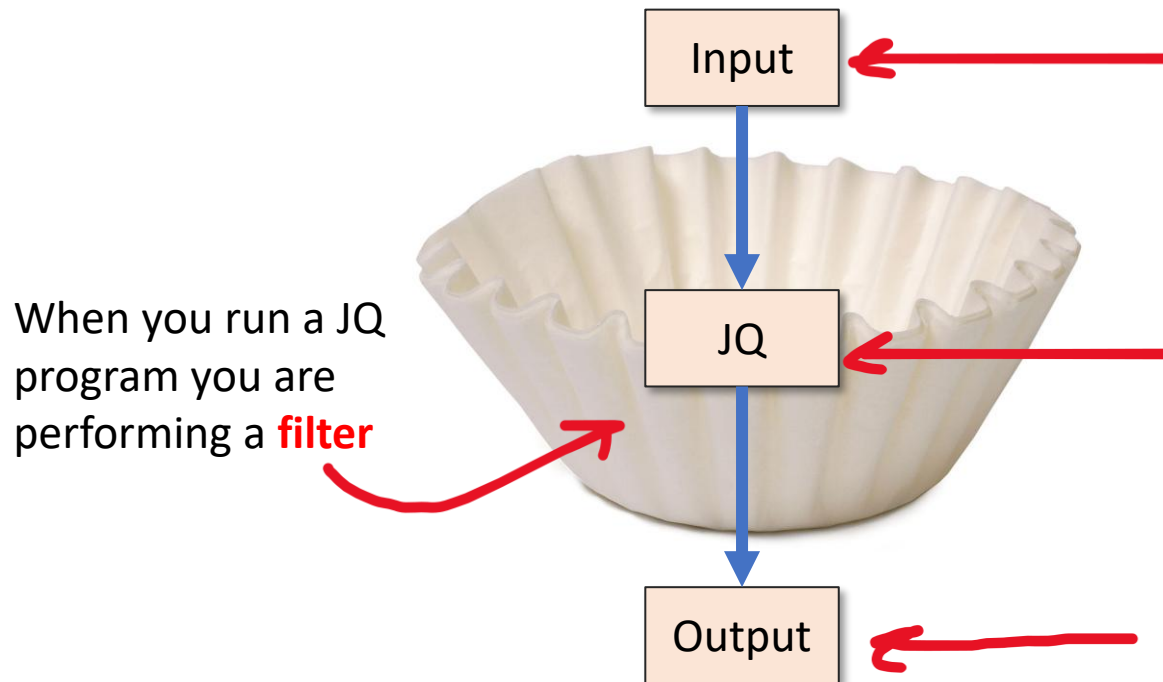
Something that self-referencing and self-aware is considered in slang to be **“meta”**

This was to distinguish YAML to be **intended as a data document** as opposed to a markup document

JQ Library

./jq

JQ is a command line tool used to **parse, format or transform a JSON response** from other commands



```
{  
  "hello": [  
    "world",  
    "mars",  
    "neptune"  
  ]  
}
```

```
echo '{ "hello": ["world","mars","neptune"] }' | jq .hello[1]
```

```
"mars"
```

using JQ's **query/filter language**

JP Library

JP is a command line tool that filters json data.
It is similar to JQ but *supposed* to be easier to use due to the JMSEPATH query language

```
aws ec2 describe-vpcs | jp .  
aws ec2 describe-vpcs | jp .Vpcs  
aws ec2 describe-vpcs | jp Vpcs  
aws ec2 describe-vpcs | jp Vpcs[*].CidrBlock
```

JP can be **installed using node package manager** as a global package

```
npm i jp -g
```



JMESPath is the same syntax that AWS client side filtering eg. **--query** so we are **stacking our knowledge**

YQ Library

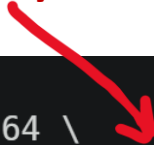
YQ is a wrapper of the JQ library (for JSON) to allow us to **filter yaml in various ways**.

```
yq -o=json '.' ./examples/yaml/hello.yaml  
[  
  "world"  
]
```

- YQ is useful when need to transform yaml data from one structure to another
- YQ uses the same JQ language to filter
- YQ has YAML specific utilities
- YQ is a good tool to validate how an input YAML is interpreted
 - YQ is a great tool to see the **JSON equivalent of our YAML**

YQ - Installing

To install you can **download the binary** from the Github repo Releases page:



```
VERSION=v4.2.0 and BINARY=yq_linux_amd64 \  
wget https://github.com/mikefarah/yq/releases/download/${VERSION}/${BINARY}.tar.gz  
-O - | tar xz && mv ${BINARY} /usr/bin/yq
```

```
brew install yq  
snap install yq
```

You can also install via the **package managers**:

- Brew
 - Snap
- 

YQ – Common Flags

YQ has the following notable flag features:

Can (and does by default) print output with colors and also pretty print



```
yq --colors --prettyPrint '.' sample.yml  
yq --no-colors '.' sample.yml
```

Can take JSON as input and produce JSON output



```
yq -P '.' sample.json  
yq -o=json '.' sample.yml
```

Can output to CSV or TSV (tabular data)



```
yq -o=csv sample.yml  
yq -o=tsv sample.yml
```

Can take XML input and produce JSON output



```
yq -p=xml -o=xml '.' sample.xml
```

Can process “Front Matter” formatted YAML



```
yq --front-matter=process '.a="chocolate"' file.jekyll
```

Can split multi-document yaml into several files



```
yq --split-exp '.a' myfile.yml
```

Shell completion can be turned on for easier use



```
source <(yq shell-completion bash
```

Control outputted space tabs size (defaulted to 2 space)



```
yq -j --indent 4 sample.yml
```

YQ – Inputting Files

There are **generally three ways** you can input a file into YQ:

```
yq file.yaml  
yq . file.yaml  
cat file.yaml | yq .
```



1. Just the filename
2. The filter along with the filename
3. **Pipe** input from another bash command

- Piping is a common use case when get output JSON from another program
- Loading a yaml file is common if there is a configuration file part of an application codebase you need to filter

YQ appears to have an embedded version of JQ.
So when you install YQ you don't have to install JQ separately

AWS CLI – Parameters from Files

Some parameters (command flags) expect file names as arguments
Some parameters will accept either inline text or alternatively a file path

```
aws ec2 describe-instances --filters file://filter.json  
aws ec2 describe-instances --filters file:///tmp/filter.js  
aws ec2 describe-instances --filters 'file://filter content.json'
```

- Relative file
- Absolute path for file
- Quote strings if you have spaces

```
aws ec2 describe-instances --filters file:///./filter.json  
aws ec2 describe-instances --filters file:///~/filter.json  
aws ec2 describe-instances --filters file:///../filter.json
```

- Explicit relative
- Home directory
- Going back a directory

For Windows Command Prompt the slashes for directories need to go the other way eg: file://mypath\file.txt

AWS CLI – Parameters from Files

Commands that take Binary data use fileb://

```
aws ec2 run-instances: --user-data fileb://user-data.sh  
aws s3api put-object: --sse-customer-key fileb://sse.key  
aws kms decrypt: --ciphertext-blob fileb://ciphertext
```

The encoding for binary to text file in Base64.

```
require "base64"  
enc = Base64.encode64('Send reinforcements')  
# -> "U2VuZCB5ZWluZm9yY2VtZW50cw==\n"
```

Base64 Encoding

Base64 is a binary to a text encoding scheme that represents binary data in an ASCII (American Standard Code for Information Interchange) string format.

Index	Binary	Char	Index	Binary	Char	Index	Binary	Char	Index	Binary	Char
0	000000	A	16	010000	Q	32	100000	g	48	110000	w
1	000001	B	17	010001	R	33	100001	h	49	110001	x
2	000010	C	18	010010	S	34	100010	i	50	110010	y
3	000011	D	19	010011	T	35	100011	j	51	110011	z
4	000100	E	20	010100	U	36	100100	k	52	110100	0
5	000101	F	21	010101	V	37	100101	l	53	110101	1
6	000110	G	22	010110	W	38	100110	m	54	110110	2
7	000111	H	23	010111	X	39	100111	n	55	110111	3
8	001000	I	24	011000	Y	40	101000	o	56	111000	4
9	001001	J	25	011001	Z	41	101001	p	57	111001	5
10	001010	K	26	011010	a	42	101010	q	58	111010	6
11	001011	L	27	011011	b	43	101011	r	59	111011	7
12	001100	M	28	011100	c	44	101100	s	60	111100	8
13	001101	N	29	011101	d	45	101101	t	61	111101	9
14	001110	O	30	011110	e	46	101110	u	62	111110	+
15	001111	P	31	011111	f	47	101111	v	63	111111	/

We convert each character to binary eg. Man

- M (77) -> 01001101
- a (97) -> 01100001
- n (110) -> 01101110

So the example represented as a stream of bytes (8 bits):

- 01001101 01100001 01101110

To encode we then read 6 bits at a time rather than 8

- 010011 010110 000101 101110



We use the **base 64 table** to map the **6 bit chunks** into the corresponding character:

- 010011 (19) -> T
- 010110 (22) -> W
- 000101 (5) -> F
- 101110 (46) -> u

And so end up with: **TW Fu**

AWS CLI – Quotes with Strings

When you are inputting parameter values like json **objects quoting will help avoid weird termination or incomplete interrupting of commands.**

You can use single quotes.

```
aws ec2 create-key-pair --key-name 'my key pair'
```

You can use double quotes within single quotes

- You don't need to escape the quotes

```
$ aws ec2 run-instances \  
  --image-id ami-12345678 \  
  --block-device-mappings ' [{"DeviceName":"/dev/sdb","Ebs":{"VolumeSize":20}} ] '
```

You can use double quotes.

```
aws ec2 describe-vpcs --region "us-east-1"
```

You **cannot** use singles in doubles

```
aws ec2 describe-vpcs --region "us-east-1" --filters "[{'Name':'state','Values': ['avaliable']}]"
```

Invalid JSON: Expecting property name enclosed in double quotes: line 1 column 3

The examples above are for Linux/Unix, PowerShell and Windows command prompt are different

AWS CLI – Common Parameter Types

Parameters (command flags) are designated a parameter types which determines its expected values format.

- String
- Timestamp
- List
- Boolean
- Integer
- Binary / blob (binary large object) and streaming blob Map Document

You can see what type each parameter is within the **AWS CLI Command Reference documentation** for the specified subcommand

`--start-time` (timestamp)

The date and time, up to the past 90 days, from the specified format (for example, `YYYY-MM-DD T*HH*`)



AWS CLI – Common Parameter Types

Strings: can contain alphanumeric characters, symbols, and white spaces

- When using white space you need to wrap in quotes

```
--bucket-name s3://my-bucket
```

Timestamps (Datetime/Date) are formatted according to the ISO 8601 standard.

Acceptable formats:

- YYYY-MM-DDThh:mm:ss.sssTZD (UTC)
- YYYY-MM-DDThh:mm:ss.sssTZD (with offset)
- YYYY-MM-DD
- Unix Epoch time eg. 1412195400

```
--start-time 2014-10-13T19:00:00Z
```

List One or more strings separated by spaces

- Strings that have space need quotes

```
--instance-types m1.xlarge m1.medium
```

AWS CLI – Skeleton Templates

The Generate CLI Skeleton allows you to generate out a json or yaml file that can be then modified with inputs, and used to mass import configuration for a future command



```
aws ec2 run-instances --generate-cli-skeleton input > ec2runinst.json
aws ec2 run-instances --generate-cli-skeleton yamlinput > ec2runinst.json
```

This command is commonly used with:

- --cli-input-json
- --cli-input-yaml

Not all commands accept both JSON or YAML for input.

- So while you can generate our YAML you may need to convert it to json for specific subcommands



```
{
  "DryRun": true,
  "ImageId": "",
  "MinCount": 0,
  "MaxCount": 0,
  "KeyName": "",
  "SecurityGroups": [
    ""
  ],
  "SecurityGroupIds": [
    ""
  ],
  "UserData": "",
```

AWS CLI – CLI Input Flag

We can populate parameters (command flags) *if* the --cli-input flag is available for a subcommand

```
aws iotevents create-input --cli-input-json file://pressureInput.json
aws iotevents create-input --cli-input-yaml file://pressureInput.yaml
```

```
create-input
--input-name <value>
[--input-description <value>]
--input-definition <value>
[--tags <value>]
[--cli-input-json | --cli-input-yaml]
[--generate-cli-skeleton <value>]
[--debug]
```

Subcommands support either one or both JSON or YAML:

- --cli-input-json
- --cli-input-yaml

Check the docs to be sure...

AWS CLI – Shorthand Syntax

Instead of writing this json
into CLI command flags



```
--option '{"key1":"value1","key2":"value2","key3":"value3"}'
```

We can use a shorthand syntax that
reduces the amount we type



```
--option key1=value1,key2=value2,key3=value3
```

Example of a JSON object

```
aws dynamodb update-table --provisioned-throughput '{"ReadCapacityUnits':15,'WriteCapacityUnits':10}'
aws dynamodb update-table --provisioned-throughput ReadCapacityUnits=15,WriteCapacityUnits=10
```

Example of an Array

```
aws ec2 stop-instances --instance-ids ['i-1486157a','i-1286157c','i-ec3a7e87']
aws ec2 stop-instances --instance-ids i-1486157a i-1286157c i-ec3a7e87
```

Example of an Array of
JSON objects

```
aws ec2 create-tags --tags '[
    {"Key": "My1stTag", "Value": "Value1"},
    {"Key": "My2ndTag", "Value": "Value2"},
    {"Key": "My3rdTag", "Value": "Value3"}
]'

aws ec2 create-tags \
    --tags Key=My1stTag,Value=Value1 Key=My2ndTag,Value=Value2 Key=My3rdTag,Value=Value3
```

PowerShell AWS to AWS CLI

By default, Windows PowerShell outputs text as UTF-16, which conflicts with the UTF-8 encoding used by JSON files and many Linux systems.

It is recommended you use **-Encoding ascii** with your PowerShell Out-File commands to ensure the AWS CLI can read the resulting file.



```
New-S3Bucket -BucketName mybucket -Region us-west-2 -Encoding ascii  
| Out-File -FilePath .\mybucket.txt
```

Chaining with Env Vars

You can export the output to a bash ENV Var by wrapping your command with `$()` eg `export VAR=$(<command>)`



```
export VPC_ID=$(awslocal ec2 describe-vpcs \
--filters 'Name=cidr,Values=10.0.0.0/16' \
--query 'Vpcs[0].VpcId' \
--output text)
```

Then you can just pass the Env Var to another command as follows:



```
aws ec2 create-subnet \
--vpc-id $VPC_ID \
--cidr-block 10.0.0.0/24 \
--tag-specifications 'ResourceType=subnet,Tags=[{Key=Name,Value=my-subnet}]')
```

Chaining with XArgs

You can use **XArgs** to pass along the output of AWS CLI command to another as a single command:



```
aws ec2 describe-vpcs --query "Vpcs[0].VpcId" --output text | \
xargs -I {} aws ec2 describe-subnets \
    --filters="Name=vpc-id,Values={}" \
    --query "Subnets[*].[CidrBlock,SubnetId]" \
    --output table
```

Xargs (extended arguments) is a unix command use to build and execute commands from standard input.

xargs is used to build commands. so if a command needs argument passed instead of input on stdin you use xargs using a simple pipe

Configurations Files

AWS has two different configurations files in TOML format at:

~/.aws/credentials – used to store sensitive credentials eg. AWS Access Key and Secret

~/.aws/config – used to store generic configurations e.g. Preferred Region

```
[default]
aws_access_key_id=AKIAIOSFODNN7EXAMPLE
aws_secret_access_key=wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
```

~/.aws/credentials

```
[default]
region=us-west-2
output=json
```

~/.aws/config

You can store all the same configuration in either files, though credentials will take precedence over config

Global Configuration options for configuration files:

- **aws_access_key_id**
- **aws_secret_access_key**
- aws_session_token
- ca_bundle
- cli_auto_prompt
- cli_binary_format
- cli_history
- cli_pager
- cli_timestamp_format
- credential_process
- credential_source
- duration_seconds
- external_id
- max_attempts
- mfa_serial
- **output**
- parameter_validation
- **region**
- retry_mode
- role_arn
- role_session_name
- source_profile
- sso_account_id
- sso_region
- sso_registration_scopes
- sso_role_name
- sso_start_url
- web_identity_token_file
- tcp_keepalive

Configurations Files – S3 Specific Settings

S3 supports several settings that configure how the AWS CLI performs Amazon S3 operations

Some apply to all S3 commands in both the s3api and s3 namespaces

The aws s3 transfer commands cp, sync, mv, and rm have additional settings you can use to control S3 transfers

```
[profile development]
s3 =
    max_concurrent_requests = 20
    max_queue_size = 10000
    multipart_threshold = 64MB
    multipart_chunksize = 16MB
    max_bandwidth = 50MB/s
    use_accelerate_endpoint = true
    addressing_style = path
```

Named Profiles

AWS Config files support the ability to have multiple profiles.

Profiles **allow you to switch better different configurations quickly** for different environments.

```
aws ec2 describe-instances
```

The default profile is what will be used when no profile flag is specified.

```
[default]
aws_access_key_id=AKIAIOSFODNN7EXAMPLE
aws_secret_access_key=wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY

[development]
aws_access_key_id=AKIAIOSFODNN7EXAMPLE
aws_secret_access_key=wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY

[production]
aws_access_key_id=AKIAIOSFODNN7EXAMPLE
aws_secret_access_key=wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
```

Use the **--profile flag** in order to change profiles per CLI run

```
aws ec2 describe-instances --profile developement
```

You can choose to not have a [default] profile.

Leaving out the default profile is a good practice against misconfiguration.

You can set an **ENV VAR** to set the profile as well.

```
export AWS_PROFILE="production"
```

AWS CLI Configure Commands

AWS CLI has multiple configure commands to make configuration easy:

```
$ aws configure
AWS Access Key ID [None]: AKIAIOSFODNN7EXAMPLE
AWS Secret Access Key [None]:
wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
Default region name [None]: us-west-2
Default output format [None]: json
```

A wizard to quickly setup your configuration file

```
aws configure set region us-west-2 --profile dev
```

Set a value for a specific setting

```
aws configure set region "" --profile dev
```

Unset a value by use the set command with a blank string

```
$ aws configure get region
ca-central-1
```

Use get to print the value of a setting

```
aws configure import --csv file://credentials.csv
```

Import AWS credentials generated and the downloaded from the AWS Console

```
aws configure list
aws configure list-profiles
```

- List out current configuration
- List of profiles

AWS CLI with SSO

If you have Single Sign ON configured in your AWS account you can configure login using the sso command:

```
$ aws configure sso
SSO session name (Recommended): my-sso
SSO start URL [None]: https://my-sso-portal.awsapps.com/start
SSO region [None]: us-east-1
SSO registration scopes [None]: sso:account:access
```

AWS CLI - Retries

When an API request fails due to possible network issues AWS CLI will automatically retry using exponential backoff.

AWS CLI can be configured to **use three different retry modes**:

Legacy retry mode

- A default of 4 for max retry attempts
- a total of 5 call attempts
- exponential backoff by a base factor of 2

Standard retry mode

- A default of 2 for max retry attempts
- a total of 3 call attempts
- exponential backoff by a base factor of 2 for a max backoff time of 20 seconds
- Supports my retries on various errors and exception over legacy

Adaptive retry mode

- experimental retry mode that includes all the features of standard mode
- Intelligently modifies the rate limit variables based on type of exception and error and pass attempts.

You can set the **retry_mode** in your configuration

```
[default]  
retry_mode = standard  
max_attempts = 6
```

You can override the **max_attempts** in the configuration

AWS CLI with EC2 metadata

Each EC2 instance contains metadata that the AWS CLI can directly use for temporary credentials

When an IAM role is attached to the instance, the AWS CLI automatically and securely retrieves the credentials from the instance metadata.

If we check the configuration file on the EC2 instance you can see its using **credentials_source**



```
[profile profilename]
role_arn = arn:aws:iam::123456789012:role/rolename
credential_source = Ec2InstanceMetadata
region = region
```

We can **turn this off** using an ENV VAR



```
export AWS_EC2_METADATA_DISABLED=false
```

AWS CLI through Proxy

When you need to access the AWS CLI via proxy server you can set the following env vars:

If you're working with a proxy you can set the following ENV VAR

```
export HTTPS_PROXY=http://proxy.example.com:5678
```

If you need it authenticated

```
export HTTPS_PROXY=http://username:password@proxy.example.com:567
```

If you are proxying on an EC2 instance you need to exempt the address used to access the instance metadata.

```
export NO_PROXY=169.254.169.254
```

AWS CLI with IAM Roles

You can configure the AWS CLI to use an IAM role instead of the default permissions of the user access keys

```
[profile prod]
role_arn = arn:aws:iam::123456789012:role/prodrole
source_profile = user1
```

role_arn:

- The role we should assume

source_profile:

- The other profile that will contain the access keys

For EC2 or ECS you can use the attach role by specifying the credential source.

credential_source:

- **Environment** – Retrieves the source credentials from environment variables.
- **Ec2InstanceMetadata** – Uses the IAM role attached to the Amazon EC2 instance profile.
- **EcsContainer** – Uses the IAM role attached to the Amazon ECS container.

Why assume a role instead of directly using roles attached to the access keys?

- **Enhance Security** – The role is decoupled from the permissions of the machine user.
- **Temporary Credentials** – assuming a role will generate STS which are short-lived instead of long-lived
- **Cross-Account Access** – you have to assume roles for cross account access

AWS CLI with External Credentials

If you need an external program or script to generate and pass along credentials you can use the **credentials_process** which takes a bash command as a input

```
[profile developer]
credential_process = /opt/bin/awscreds-custom --username andrew
```

This is a script or program you have written (or provided)

```
{
  "Version": 1,
  "AccessKeyId": "an AWS access key",
  "SecretAccessKey": "your AWS secret access key",
  "SessionToken": "the AWS session token for temporary credentials",
  "Expiration": "ISO8601 timestamp when the credentials expire"
}
```

The program is expected to output into STDOUT the following JSON which will get passed to the CLI

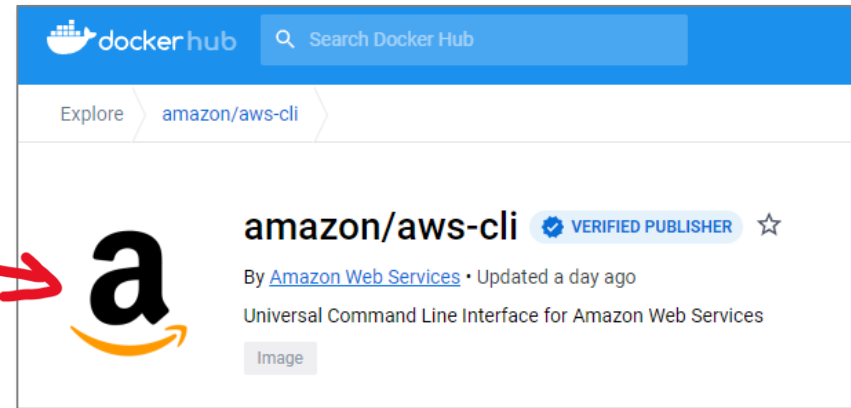


It is not recommended by AWS to use an external program to generate credentials and could pose a security risk (unless you know what you're doing). Exercise good judgement.

AWS CLI – Docker Image

AWS CLI has an official Docker Image so you can execute AWS CLI command in an isolated container

<https://hub.docker.com/r/amazon/aws-cli>



How to **execute** the AWS CLI using docker

```
docker run --rm -it amazon/aws-cli <command>
```

- It needs to download the container once.
- It will output the results in the terminal

```
gitpod /workspace/aws-cli-labs (main) $ docker run --rm -it amazon/aws-cli --version
Unable to find image 'amazon/aws-cli:latest' locally
latest: Pulling from amazon/aws-cli
68028ec3b506: Pull complete
eb2e36a17548: Pull complete
b526055860f6: Pull complete
035048f12ad7: Pull complete
fa1146dace52: Pull complete
Digest: sha256:aeb8e6a231781a70d2a37377a0e81404e40565bdbf52d26204cf433e12641801
Status: Downloaded newer image for amazon/aws-cli:latest
aws-cli/2.9.0 Python/3.9.11 Linux/5.15.0-47-generic docker/x86_64.amzn.2 prompt/off
gitpod /workspace/aws-cli-labs (main) $
```

AWS CLI – Docker Image

You can use the official docker container in order to run AWS CLI commands

```
docker run --rm -it public.ecr.aws/aws-cli/aws-cli:latest ec2 describe-vpcs
```

There are a couple of challenges:

- How do you pass along credentials?
- How do download files?

Passing along Credentials

```
docker run --rm -it \  
-v ~/.aws:/root/.aws \  
public.ecr.aws/aws-cli/aws-cli \  
s3 ls
```

- We map a volume for the local .aws folder
- Would passing Env Vars work? -e POSTGRES_USER='postgres'

Downloading Files

```
docker run --rm -it \  
-v ~/.aws:/root/.aws \  
-v $(pwd):/aws \  
public.ecr.aws/aws-cli/aws-cli \  
s3 cp s3://aws-cli-docker-demo/hello .
```

- We map a volume for the files to download to the current current working directory

AWS CLI – Installing / Updating

AWS CLI can be **installed** on:

- Windows
- macOS
- **Linux**

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
unzip awscliv2.zip
sudo ./aws/install
```

Check the AWS Documentation for your OS's configuration

Updating on Linux

```
sudo ./aws/install --bin-dir /usr/local/bin --install-dir /usr/local/aws-cli --update
```

There are third-party installations not-maintained by AWS
eg. `brew install awscli` Its not recommended to use these third-party libraries because there is no guarantee they are up-to-date. Always use AWS Docs Installation guide

Its not “officially” maintained by AWS

awscli

Install command: `$ brew install awscli`

Also known as: **awscli@2**

Official Amazon AWS command-line interface

<https://aws.amazon.com/cli/>

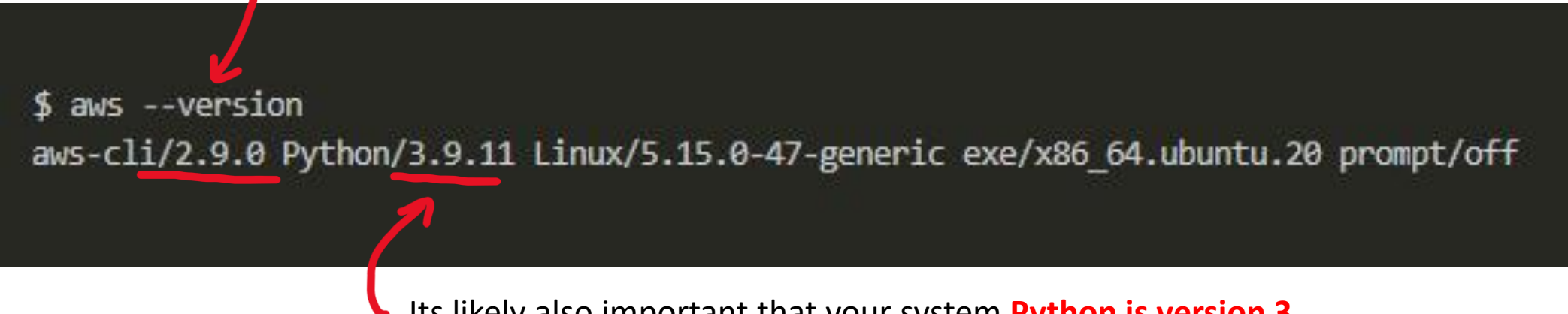
AWS CLI – Version History

AWS CLI has two versions:

- v2 – The latest version
- v1 – The old version

There is no reason to use the old version of the AWS CLI

Use the **--version** flag to determine your current version



```
$ aws --version
aws-cli/2.9.0 Python/3.9.11 Linux/5.15.0-47-generic exe/x86_64.ubuntu.20 prompt/off
```

Its likely also important that your system **Python is version 3**.

- AWS CLI is written in the Python programming language

AWS CLI – Service Models

AWS CLI underneath is using botocore (AWS SDK for Python).
All the possible **operations are defined in json files** and are loaded at runtime. These json files are called “models”



```
"DescribeVpcs":{
  "name":"DescribeVpcs",
  "http":{
    "method":"POST",
    "requestUri":"/"
  },
  "input":{"shape":"DescribeVpcsRequest"},
  "output":{"shape":"DescribeVpcsResult"},
  "documentation":"<p>Describes one or more of your VPCs.</p>"
},
```

All the Service Models json files can be found in the **botocore github repo**:

<https://github.com/boto/botocore/tree/develop/botocore/data>

You can load your own custom models, overriding existing models

```
aws configure add-model --service-model file:///var/tmp/polly.json
ls -l ~/.aws/models/polly/2016-06-10/service-2.json
```

A custom model allows you to:

- Change the behaviour of the AWS CLI
- Update just one specific service

AWS CLI – MFA

AWS CLI can make use of MFA to protect your credentials.
In order to use MFA you need to create a temporary session

How it works depends varies the process:

- Hardware MFA - GAHT12345678
- Virtual MFA - arn:aws:iam::123456789012:mfa/user

AWS CLI – MFA with Role

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "",
      "Effect": "Allow",
      "Principal": { "AWS": "arn:aws:iam::123456789012:user/anika" },
      "Action": "sts:AssumeRole",
      "Condition": { "Bool": { "aws:multipactorAuthPresent": true } }
    }
  ]
}
```

Modify the trust relationship on the IAM role to require MFA via a condition.

```
[profile role-with-mfa]
region = us-west-2
role_arn= arn:aws:iam::128716708097:role/cli-role
source_profile = cli-user
mfa_serial = arn:aws:iam::128716708097:mfa/cli-user
```

We'll see the **MFA_SERIAL** which could be an ARN or a serial number of a hardware MFA

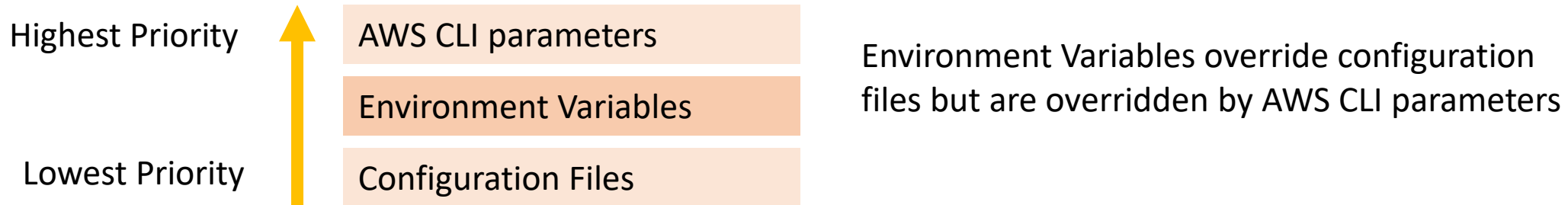
```
$ aws iam list-users --profile role-with-mfa
Enter MFA code for arn:aws:iam::123456789012:mfa/cli-user:
{
  "Users": [
    {
      ...
    }
  ]
}
```

When you attempt a command it will prompt you to enter your MFA code...

AWS CLI – Environment Variables

Environment variables (env vars) provide a way to specify AWS CLI configuration and credentials

This is **useful for scripting** or temporarily setting a profile as default



Common Environment Variables for AWS CLI

- **AWS_ACCESS_KEY_ID** – The Access Key for **programmatic access**
- **AWS_SECRET_ACCESS_KEY** – The Secret Key for **programmatic access**
- **AWS_DEFAULT_REGION** – The default region to be used when no region is specified
- **AWS_REGION** – Overrides **AWS_DEFAULT_REGION** and profile settings
- **AWS_PROFILE** - AWS CLI profile name with credentials

AWS CLI – Environment Variables

AWS CLI Environment Variable options:

- **AWS_CA_BUNDLE** – Path to custom **certificate bundle** for HTTPS validation
- **AWS_CLI_AUTO_PROMPT** - Enables the auto-prompt for AWS CLI v2
- **AWS_CLI_FILE_ENCODING** - Encoding for text files
- **AWS_CONFIG_FILE** - Path to AWS CLI **configuration file** (the default path is ~/.aws/config)
- **AWS_DEFAULT_OUTPUT** – Overrides profile **output format** setting
- **AWS_SHARED_CREDENTIALS_FILE** – Path to AWS **credentials file**

Role and Session Environment Variables

- **AWS_ROLE_ARN** - the Amazon Resource Name (ARN) of an **IAM role** with a web identity provide
- **AWS_ROLE_SESSION_NAME** - Custom session name (auto-generated if not specified)
- **AWS_SESSION_TOKEN** - Token for **temporary security credentials** from **AWS STS**
- **AWS_WEB_IDENTITY_TOKEN_FILE** - the path to OAuth 2.0 or **OpenID Connect ID token**

AWS CLI – Environment Variables

Metadata & Retry Settings

- **AWS_EC2_METADATA_DISABLED** - Disables Amazon **EC2 instance metadata service**
- **AWS_METADATA_SERVICE_NUM_ATTEMPTS** - Number of attempts to retrieve **instance metadata credentials**
- **AWS_METADATA_SERVICE_TIMEOUT** – **Timeout (seconds)** for instance metadata service connection

- **AWS_MAX_ATTEMPTS** - Max retry attempts for **AWS CLI requests**
- **AWS_PAGER** – Defines **output pagination program**
- **AWS_RETRY_MODE** – Retry mode for **AWS CLI requests**

AWS CLI – Autocompletion Options

AWS has multiple competing projects that provide autocompletion while typing out AWS CLI commands.

Legacy

AWS Completer

- The original auto completer for AWS.
- Most likely intended for AWS CLI v1

Defunct Project

AWS Shell

- An interactive shell with the AWS CLI
- This project has not been active since 2020 and it looks like its functionality has been directly rolled into the AWS CLI.
- Bugs that display on every call going back for 2 years
- Not recommended for use (despite being listed prominently on AWS CLI marketing page)



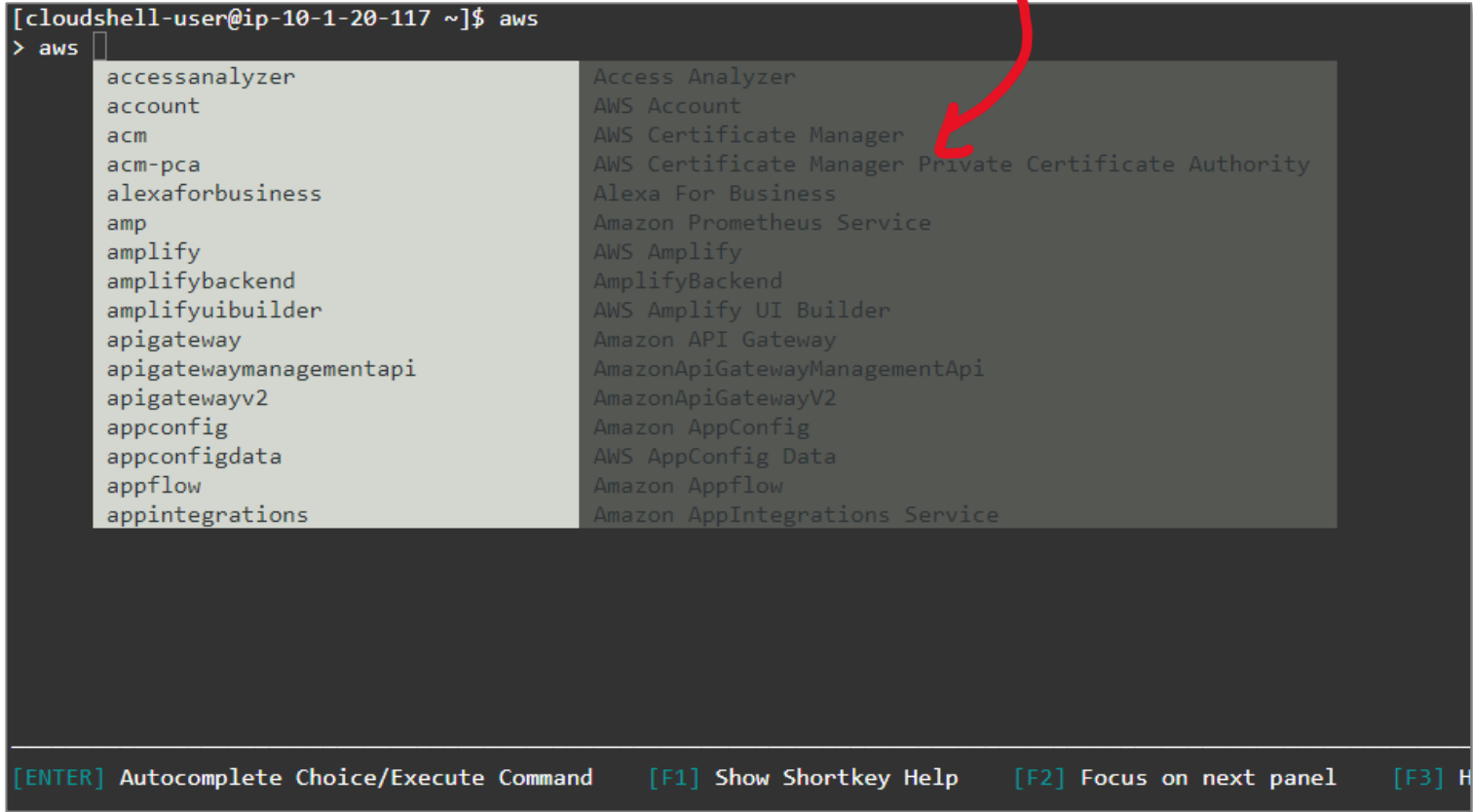
Recommended

AWS CLI Auto prompt

AWS CLI v2 can turn on auto prompt to give similar functionality of AWS Shell, the current recommended way to use autocomplete with AWS CLI

AWS CLI – Autoprompt

AWS CLI Autoprompt is a powerful **interactive shell** built into AWS CLI **to assist writing CLI commands**



```
[cloudshell-user@ip-10-1-20-117 ~]$ aws
> aws
accessanalyzer      Access Analyzer
account             AWS Account
acm                 AWS Certificate Manager
acm-pca             AWS Certificate Manager Private Certificate Authority
alexaforbusiness    Alexa For Business
amp                 Amazon Prometheus Service
amplify             AWS Amplify
amplifybackend      AmplifyBackend
amplifyuibuilder    AWS Amplify UI Builder
apigateway          Amazon API Gateway
apigatewaymanagementapi AmazonApiGatewayManagementApi
apigatewayv2        AmazonApiGatewayV2
appconfig           Amazon AppConfig
appconfigdata       AWS AppConfig Data
appflow             Amazon Appflow
appintegrations     Amazon AppIntegrations Service
```

[ENTER] Autocomplete Choice/Execute Command [F1] Show Shortkey Help [F2] Focus on next panel [F3] H

AWS CLI Autoprompt features:

- Fuzzy Search
- Command Completion
- Parameter Completion
- Resource Completion
- Shorthand Completion
- File Completion
- Region Completion
- Profile Completion
- Documentation Panel
- Projected Output Panel
- Command History
- Two different modes of activation
- Works anywhere AWS CLI is installed

AWS CLI Autoprompt – Documentation Panel

Documentation Panel will show relevant documentation for the current command or subcommand

```
> aws ec2
```

Amazon Elastic Compute Cloud (Amazon EC2) provides resizable computing capacity in the Amazon EC2 eliminates the need to invest in hardware up front. You can develop and deploy applications on Amazon EC2 instances that are powered by Amazon Elastic Compute Cloud (Amazon VPC) enables you to provision a private, secure section of the Amazon Web Services Cloud to host your Amazon Web Services resources in a virtual private cloud. Amazon Elastic Block Store (Amazon EBS) provides persistent block-level volumes for use with EC2 instances. Amazon EBS volumes are durable and reliable storage volumes that can be attached to an instance and used like a hard drive.

To learn more, see the following resources:

[ENTER] Autocomplete Choice/Execute Command

> aws ec2 describe-instances

Describes the specified instances or all instances.

If you specify instance IDs, the output includes information for only the specified instances. If you specify filters, the output includes information for only those instances that meet the filter criteria. If you do not specify instance IDs or filters, the output includes information for all instances, which can affect performance. We recommend that you use pagination to ensure that the operation returns quickly and successfully.

If you specify an instance ID that is not valid, an error is returned. If you specify an instance that you do not own, it is not included in the output.

[ENTER] Autocomplete Choice/Execute Command [F1] Show Shortkey Help [F2] Focus on next panel

- Pressing [F3] will bring up a documentation pane
- As we type and add subcommands the documentation will contextually change
- We can navigate the docs by toggle panes by pressing [F2]
 - We can navigate the text with VI directions eg. j = down k = up

AWS CLI Autoprompt – Configuration

AWS CLI Autoprompt is not turned on by default.
There are multiple ways to can enable Autoprompt

To turn on AWS CLI Autoprompt **for a single command execution** you can use the `--cli-auto-prompt` command flag



```
aws --cli-auto-prompt
```

You can turn on AWS CLI Autoprompt **for every future run** by using the `AWS_CLI_AUTO_PROMPT` Environment Variable



```
export AWS_CLI_AUTO_PROMPT=on-partial
```

You can turn on AWS CLI Autoprompt **for every future run** by setting the option within your AWS CLI Config file



```
cli_auto_prompt = on
```

AWS CLI Autoprompt - Modes

AWS CLI Autoprompt has two different modes you can enable

Full Mode

Whenever you enter an AWS CLI command into the terminal it will always activate Autoprompt mode



```
AWS_CLI_AUTO_PROMPT=on
```

Partial Mode



Recommended

Only uses auto-prompt mode if the command is incomplete or cannot run due to client-side validation errors



```
AWS_CLI_AUTO_PROMPT=on-partial
```

If you are using full mode, lets say you **execute a bash script than autoprompt will still activate** for each and every command.

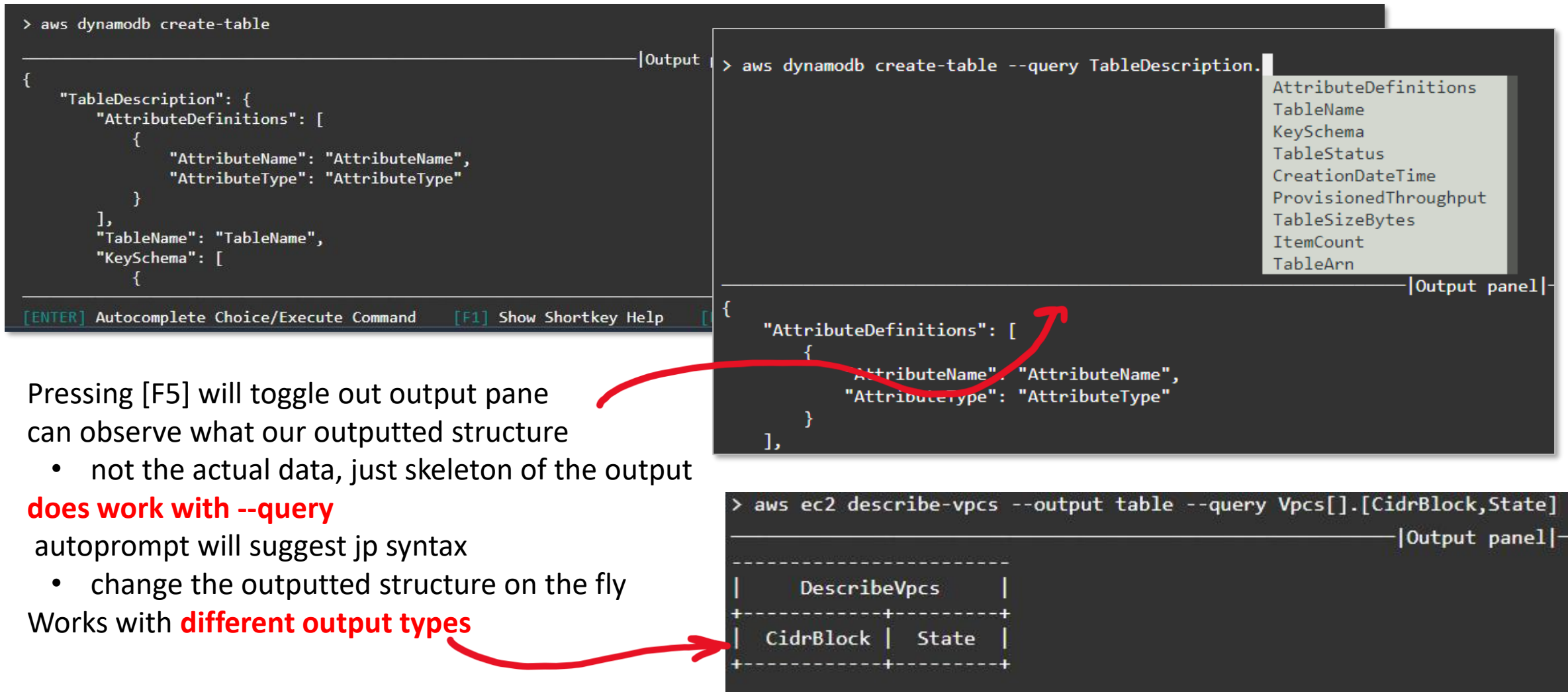
This could disrupt your workflow (unless you want to use autoprompt as a way to review before executing)



```
[cloudshell-user@ip-10-1-20-117 ~]$ ./my-script.sh
> aws ec2 describe-instances
--filters
--instance-ids
--dry-run
--no-dry-run
--max-results
--next-token
--cli-input-json
[list] The filters. affinity -
[list] The instance IDs. Default:
[boolean] Checks whether you have
[boolean] Checks whether you have
[integer] The maximum number of re
[string] The token to request the
[string] Reads arguments from the
```

AWS CLI Autoprompt – Output Panel

Output panel allows you to preview the schema of the output for a subcommand



```
> aws dynamodb create-table
```

```
{
  "TableDescription": {
    "AttributeDefinitions": [
      {
        "AttributeName": "AttributeName",
        "AttributeType": "AttributeType"
      }
    ],
    "TableName": "TableName",
    "KeySchema": [
      {

```

[ENTER] Autocomplete Choice/Execute Command [F1] Show Shortkey Help

AttributeDefinitions
TableName
KeySchema
TableStatus
CreationDateTime
ProvisionedThroughput
TableSizeBytes
ItemCount
TableArn

```
{
  "AttributeDefinitions": [
    {
      "AttributeName": "AttributeName",
      "AttributeType": "AttributeType"
    }
  ],

```

[ENTER] Autocomplete Choice/Execute Command [F1] Show Shortkey Help

```
> aws ec2 describe-vpcs --output table --query Vpcs[][CidrBlock,State]
```

```
-----
| DescribeVpcs |
+-----+-----+
| CidrBlock | State |
+-----+-----+
```

- Pressing [F5] will toggle out output pane
- can observe what our outputted structure
 - not the actual data, just skeleton of the output
- **does work with --query**
- autoprompt will suggest jp syntax
 - change the outputted structure on the fly
- Works with **different output types**

AWS CLI Autoprompt – Command History

When you are within AWS CLI Autoprompt you can press “Ctrl+R” for **previous AWS CLI history**

```
> aws  
ec2 describe-subnets --query "Subnets[*].[CidrBlock,SubnetId]" --output table --filter name=tag:Name  
ec2 describe-subnets --query "Subnets[*].[CidrBlock,SubnetId]" --output table  
s3 ls --output table  
s3 ls  
sts get-caller-identity
```

- It will show both successful and unsuccessful execute commands
- AWS CLI commands that does not enter (activate) the autoprompt mode will not show up in the history

```
(reverse-i-search)`aws ': aws ec2 describe-subnets --query "Subnets[*].[CidrBlock,SubnetId]" --output table
```

- For commands execute outside the autoprompt mode you can do “Ctrl+R” and start fuzzy typing a previous command or hit up and down on the keyboard
 - This is native feature built into Linux bash terminals, its not an AWS CLI feature.

AWS CLI Autoprompt – Autocompletion

Command completion

- suggests commands
- suggest subcommands

```
> aws sage
sagemaker
sagemaker-a2i-runtime
sagemaker-edge
sagemaker-featurestore-r
sagemaker-runtime
workmailmessageflow
```

```
> aws dynamodb
batch-execute-stat
batch-get-item
batch-write-item
create-backup
create-global-tab
```

Parameter completion

- suggest parameters (command flags)
- Indicates required parameters first

```
> aws dynamodb describe-limits
--cli-input-json [string] Reads argument
--cli-input-yaml [string] Reads argument
--generate-cli-skeleton [string] Prints a JSON
--debug [boolean] Turn on debug
--endpoint-url [string] Override comm
--no-verify-ssl [boolean] By default,
--no-paginate [boolean] Disable auto
--output [string] The formatin
```

```
> aws dynamodb create-table
--attribute-definitions (required) [list] An array o
--table-name (required) [string] The name
--key-schema (required) [list] Specifies
--local-secondary-indexes [list] One or mor
--global-secondary-indexes [list] One or mor
--billing-mode [string] Controls
--provisioned-throughput [structure] Repre
--stream-specification [structure] The s
```

Resource completion

- suggest resources in your AWS account
- You need to have credentials configured in order for resource completion to work

```
> aws dynamodb describe-table --table-name
Blog-nqwqggk5frglvt
Blog-rcqo3kplungk5c
Comment-nqwqggk5frg
Comment-rcqo3kplungk
MyTable
Post-nqwqggk5frglvt
Post-rcqo3kplungk5c
andrew-bar-raiser-p
exampro-my-app-debu
prod-sst-example-Cc
```

```
> aws ec2 describe-vpcs --vpc-ids
vpc-00e
vpc-c3b
```

AWS CLI Autoprompt – Autocompletion

Shorthand completion

- Will help autocomplete shorthand json syntax
- You can tell if parameter type is “structure”

```
> aws ec2 describe-vpcs --filters
Name= [string] The name of the filter
Values= [list] The filter values. Filter values are a list of strings.
```

```
> aws ec2 run-instances --iam-in
--iam-instance-profile [structure] The name of the IAM instance profile.
--instance-market-options [structure] The market options for the instance.
--private-dns-name-options [structure] The private DNS name options for the instance.
```

File completion

- Will autocomplete files names
- You need to type “file:///” first to see suggestions

```
> aws ec2 run-instances --cli-input-json file:///bas
bastion.json
```

Region completion

- Autocompletes region
- May not work until you type in other required parameters
 - S3 needs to know if the bucket name is possible before it autocomplete regions

```
> aws s3 mb --bucket-name rock32525 --region
af-south-1
ap-east-1
ap-northeast-1
ap-northeast-2
ap-northeast-3
ap-south-1
ap-southeast-1
ap-southeast-2
ca-central-1
```

```
> aws s3 mb --region
```

AWS CLI Autoprompt – Autocompletion

Profile completion

- Autocompletes profile information configured in the ~/.aws/credentials

```
> aws --profile development
production
```

Fuzzy searching

- You can type part of a word and autocomplete will show possible options that match
 - Notice that the suggestions here do not start with “east” so it has fuzzy on the front of the word. eg. “*east*”
 - Notice we skipped “e-” so it “m*ca*”

```
> aws dynamodb list-tables --region east
ap-east-1
sa-east-1
us-east-1
us-east-2
ap-northeast-1
ap-northeast-2
ap-northeast-3
ap-southeast-1
ap-southeast-2
ap-southeast-3
```

```
> aws dynamodb list-tables --region mca
me-central-1
```